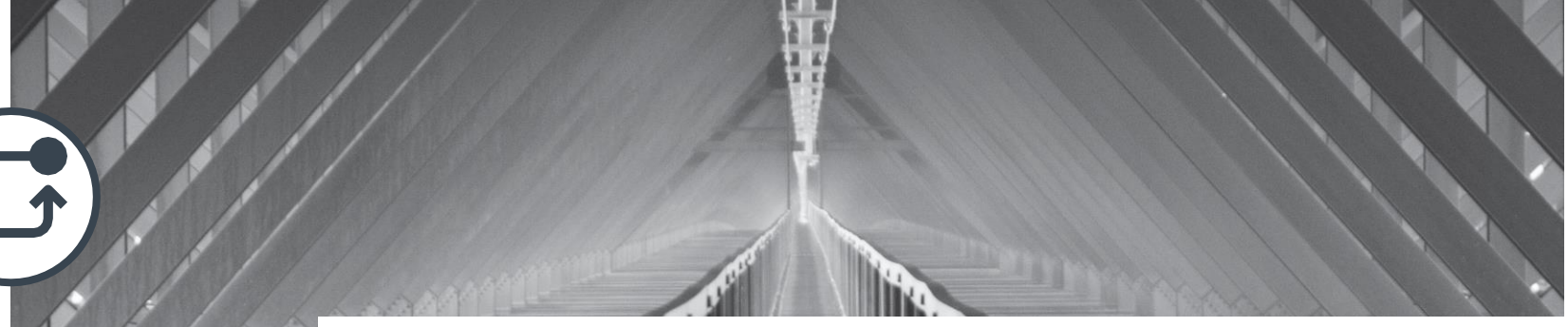
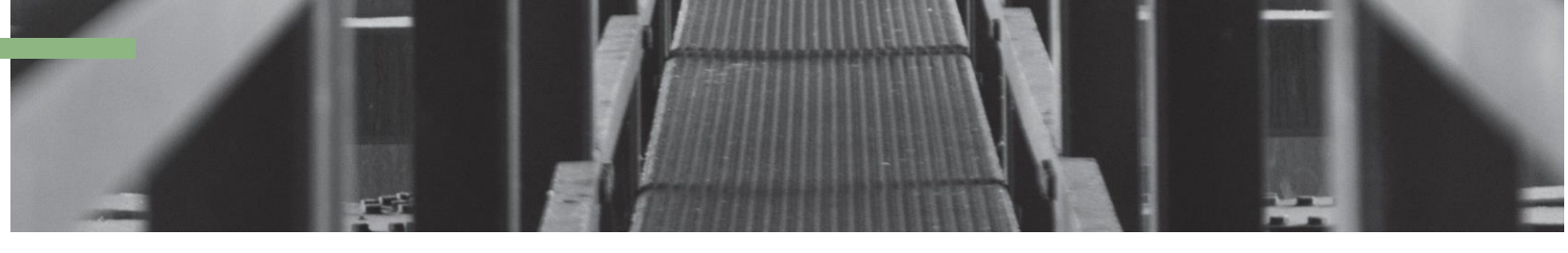


INTRODUCCIÓN A LAS BASES DE DATOS



BIENVENIDA

Bienvenido(a) a la asignatura *Introducción a las Bases de Datos*, con la cual podrás aprender lo que se requiere para crear una base de datos desde cero, de acuerdo con el diseño deseado. A su vez, adquirirás los conocimientos necesarios para crear consultas de diversos tipos.





UNIDAD 1

DISEÑO DE BASES DE DATOS



TEMARIO

1.1 MODELO ENTIDAD RELACIÓN

1.2 NORMALIZACIÓN



INTRODUCCIÓN

La asignatura de *Introducción a las Bases de Datos* tiene como objetivo apoyarte a aprender cómo se deben crear bases de datos desde cero.

En esta primera unidad aprenderás a desarrollar los modelos de entidad-relación y la normalización.

COMPETENCIAS A DESARROLLAR

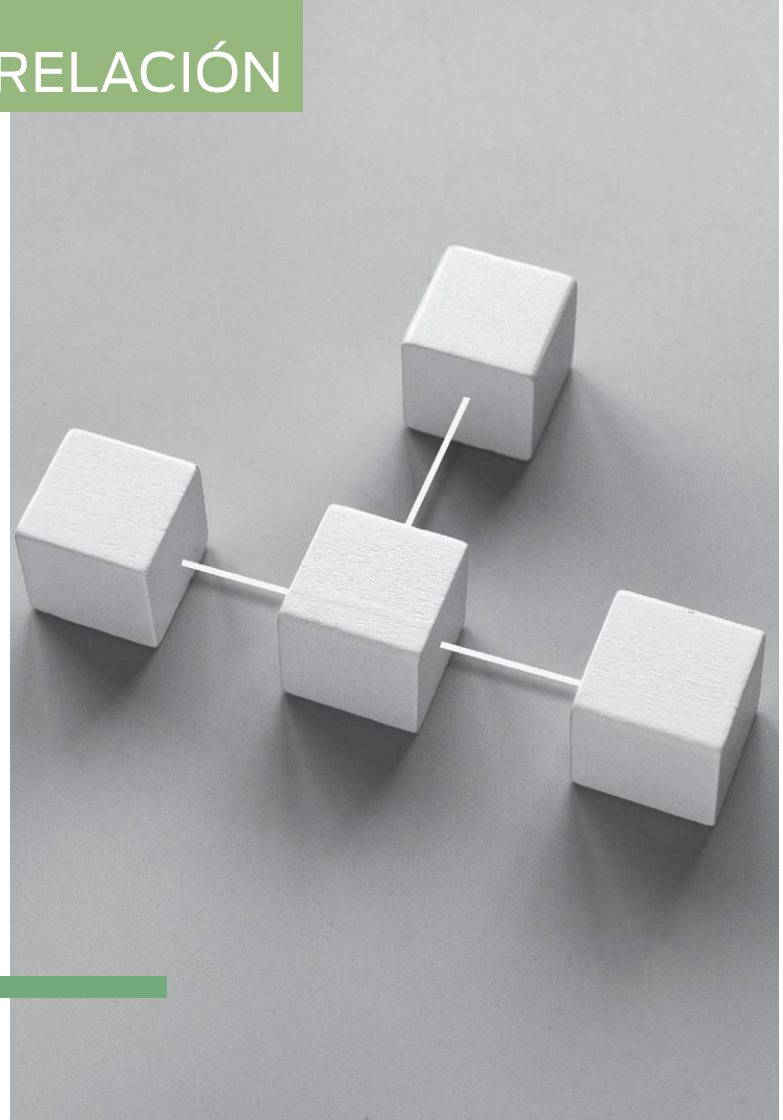


- El alumno será capaz de identificar los modelos entidad-relación.
- El alumno será capaz de identificar la normalización en bases de datos.

MODELO ENTIDAD RELACIÓN

Las bases de datos son colecciones de información diseñadas para agregar significado y eficiencia a una encuesta, estudio organizado o estructura de datos de una organización.

Son de gran importancia para las empresas y se han convertido en las últimas décadas en un componente elemental de los sistemas de información. Por regla general, los datos permanecen allí durante varios años sin tener que cambiar su composición.



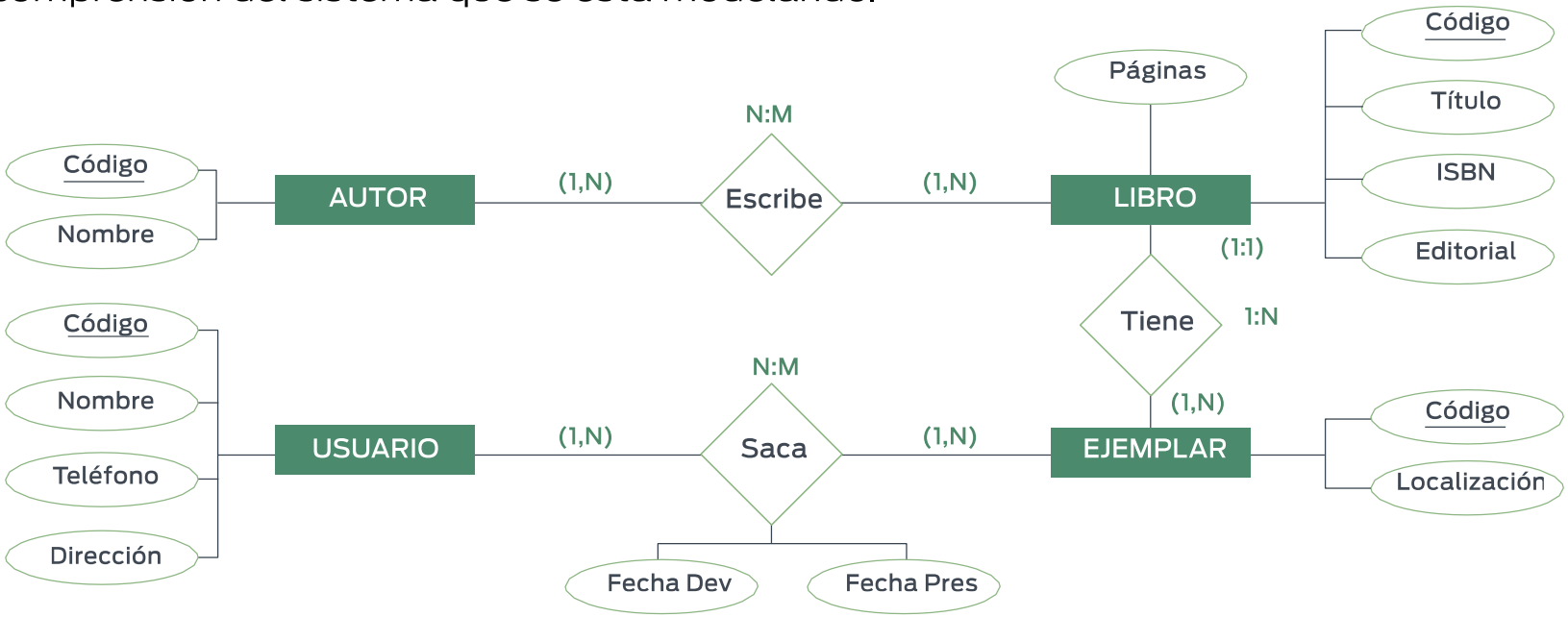
VIDEO



Te invitamos a ver el siguiente video:

Modelo Entidad - Relación

Es una herramienta de modelado de datos utilizada para describir de manera abstracta las entidades y relaciones entre ellas en un sistema o aplicación. Este modelo representa visualmente los elementos y las relaciones que existen entre ellos, permitiendo una mejor comprensión del sistema que se está modelando.

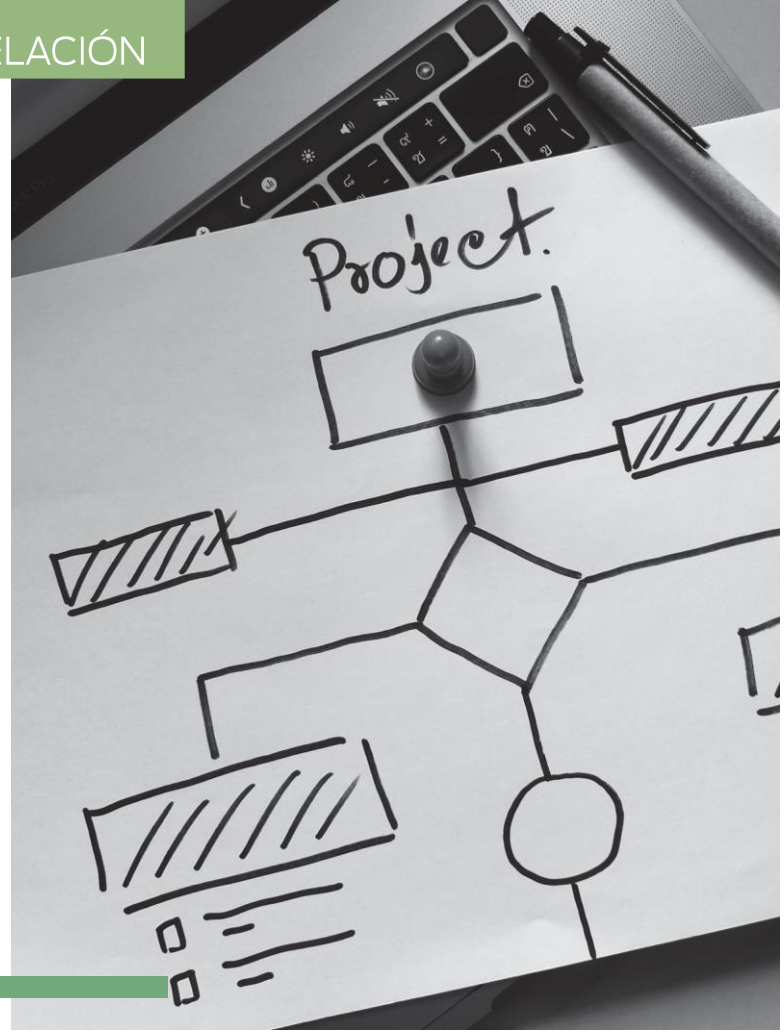


Elementos básicos de un modelo entidad relación:

1. Entidades

Cualquier tipo de objeto o criterio sobre el que se recoge información: cosas, personas, criterios abstractos o eventos. Las entidades se representan gráficamente mediante rectángulos y sus nombres aparecen dentro de ellos. Los nombres de las entidades solo pueden aparecer una vez en el esquema conceptual.

Hay 2 tipos de entidades: **fuertes y débiles.**





Débiles: Su vida es dependiente de la existencia de ejemplares de otras entidades diversas y se representan con un rectángulo de trazo doble.

Para cada atributo existe un rango de valores permitidos conocido como **Dominio** del atributo.

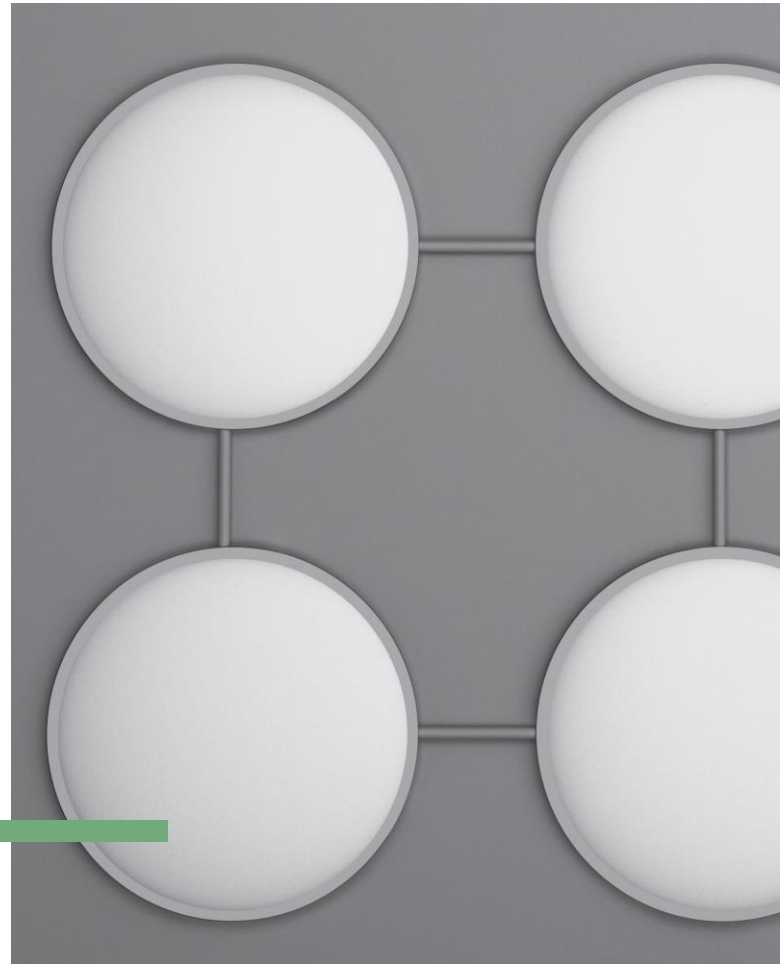
De esta forma, por ejemplo, el dominio del atributo *carnet* es cada una de las cadenas de letras y números de 7 cifras.

2. Atributo

Es la propiedad de un interés o hecho sobre una entidad o interacción. Los atributos representan características simples de entidades y relaciones. Se representan mediante pequeñas bolitas que cuelgan de la entidad a la que pertenecen.

Por ejemplo, una entidad alumno se escribiría: {(**id alumno**, 23), (**nombre,** Jenny), (**apellidos,** Cruz Suárez), (**carnet,** 8624659), (**nivel,** I)}

Existe una clase de atributos denominados **Derivados** cuyo valor se puede obtener a partir de otros atributos.





Por ejemplo, en una entidad *Factura* se podría tener un atributo *valor total* el cual se calcularía desde la sumatoria de los valores de los atributos *precio producto* multiplicado por cantidad de productos.

Por tanto, la **base de datos** es un conjunto de grupos de entidades, cada uno de los cuales tiene cualquier número de entidades del mismo tipo.

En el ejemplo que primero se manejará, se van a tener los siguientes conjuntos de entidades para una base de datos de calificaciones de una universidad.

ENTIDAD	DESCRIPCIÓN	ATRIBUTOS
Alumno	El conjunto de los estudiantes de una universidad determinada.	ID alumno carnet apellidos nombres nivel
Materia	El conjunto de las materias que se dictan en la universidad.	ID materia código materia créditos
Carrera	El conjunto de los programas académicos que se dictan en la universidad.	ID carrera carrera
Periodo	El conjunto de los periodos académicos que han transcurrido en la universidad.	ID programa periodo

En la siguiente imagen se observan los conjuntos de entidades alumno y carrera.

Alumno

1	LÓPEZ PÉREZ	JORGE	9632589	2
2	AGUIRRE CRUZ	FRANCISCO	9685748	3
3	VERA LARA	MARÍA	9855852	3
4	PÉREZ DÍAZ	ANA	9855221	4
5	GÓMEZ CASTRO	JUAN	9858585	8
6	CRUZ JUÁREZ	PEDRO	9858585	7

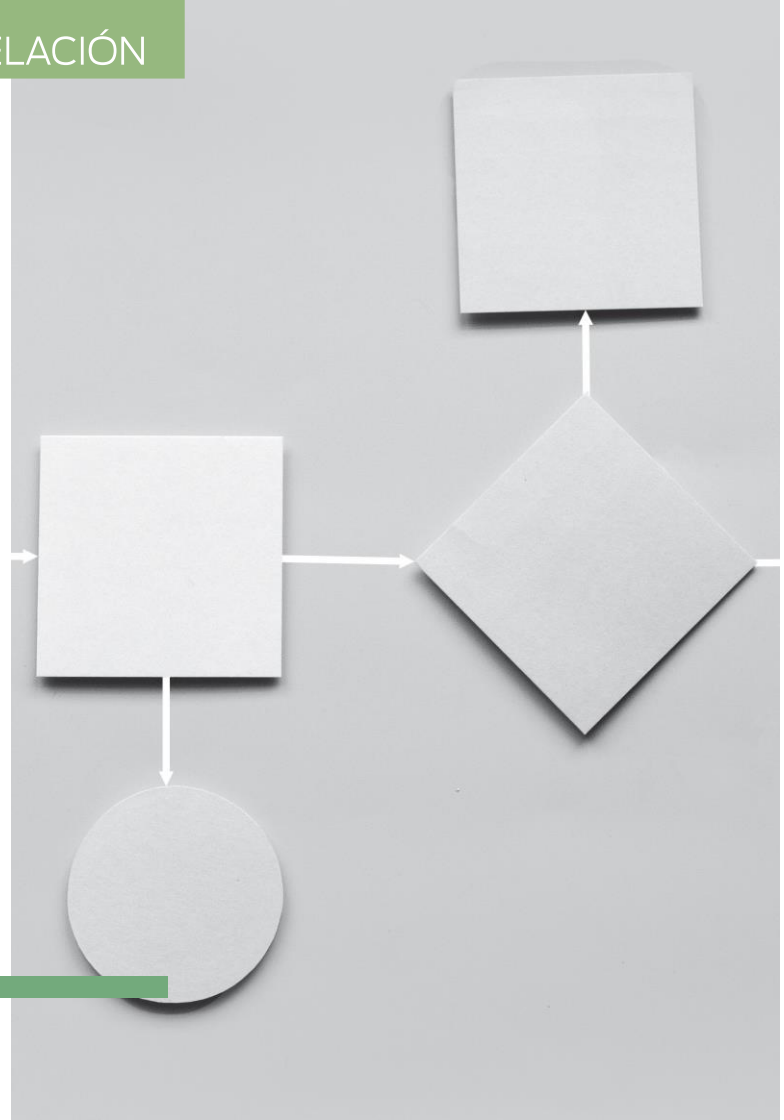
Carrera

1	INGENIERÍA INDUSTRIAL
2	CONTADURÍA

3. Relaciones

Es una correspondencia o agrupación entre 2 o más entidades. Cada interacción tiene un nombre que explica su funcionalidad. Las relaciones están representadas gráficamente por diamantes y sus nombres aparecen en ellos.

Por ejemplo, se puede conceptualizar una relación que asocie al alumno “ANA CRUZ” con la carrera de “CONTADURÍA”, lo que indicaría que el alumno está matriculado en aquella carrera. Llamaríamos al conjunto de tales relaciones *Alumno Carrera*.



Un conjunto de relaciones es una agrupación de relaciones del mismo tipo.

Para la situación de la imagen anterior se ilustraría la relación *Alumno Carrera* de la siguiente forma:

Alumno

1	LÓPEZ PÉREZ	JORGE	9632589	2
2	AGUIRRE CRUZ	FRANCISCO	9685748	3
3	VERA LARA	MARÍA	9855852	3
4	PÉREZ DÍAZ	ANA	9855221	4
5	GÓMEZ CASTRO	JUAN	9858585	8
6	CRUZ JUÁREZ	PEDRO	9858585	7

Carrera

1	INGENIERÍA INDUSTRIAL
2	CONTADURÍA

La interacción *Alumno Carrera* es un ejemplo de relación **Binaria**, lo que quiere mencionar que involucra a dos entidades. Además, tienen la posibilidad de existir relaciones que involucran a más de dos entidades, o las que se otorgan entre la misma entidad llamadas **Recursivas**.

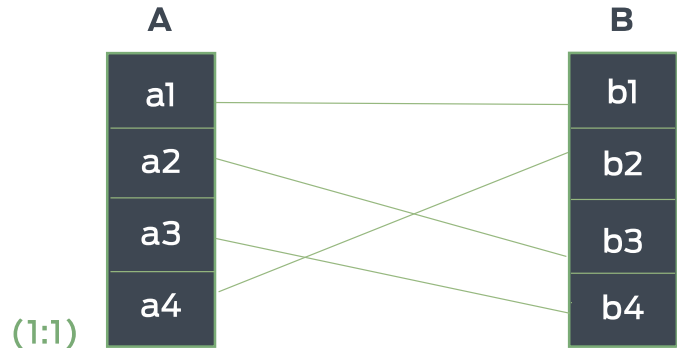
Cardinalidades de mapeo

Es el número de entidades con las que se puede asociar otra entidad a través de una interacción binaria; la cardinalidad puede ser: uno a uno, uno a muchos o muchos a uno y muchos a muchos.

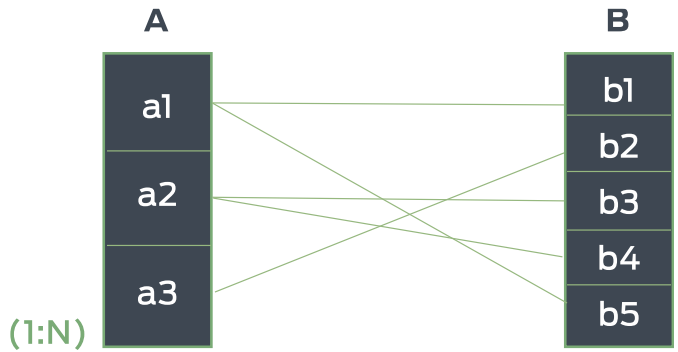
El tipo de potencia se representa mediante una etiqueta de interacción en idioma extranjero, respectivamente: “1:1”, “1:N” y “N:M”, aunque la notación depende de la utilizada, siendo el unificado la más utilizada actualmente.

Para un grupo de interacción binaria R entre los conjuntos de entidades A y B, la cardinalidad del mapa debe ser una de las siguientes:

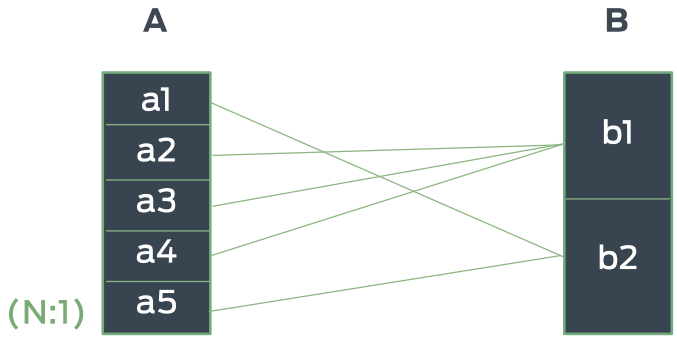
1:1 Un objeto en A solo puede relacionarse con un objeto en B solo puede relacionarse con un objeto en A.



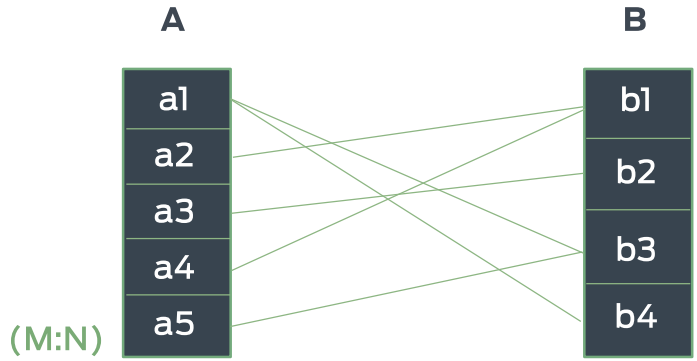
1:N La entidad A está asociada con cualquier cantidad de objetos B, pero el objeto B solo puede estar asociado con un objeto A.



N:1 Una entidad en A está relacionada solo con una entidad en B, pero una entidad en B puede estar relacionada con cualquier número de entidades en A.



Muchos a muchos: una entidad en A está relacionada con cualquier cantidad de entidades en B, y una entidad en B puede estar asociada con cualquier cantidad de entidades en A.



En el ejemplo anterior, el de *Alumno Carrera* es de cardinalidad *muchas a una* ya que el alumno puede elegir sólo una carrera, y a su vez, en una carrera pueden estar inscritos varios alumnos.

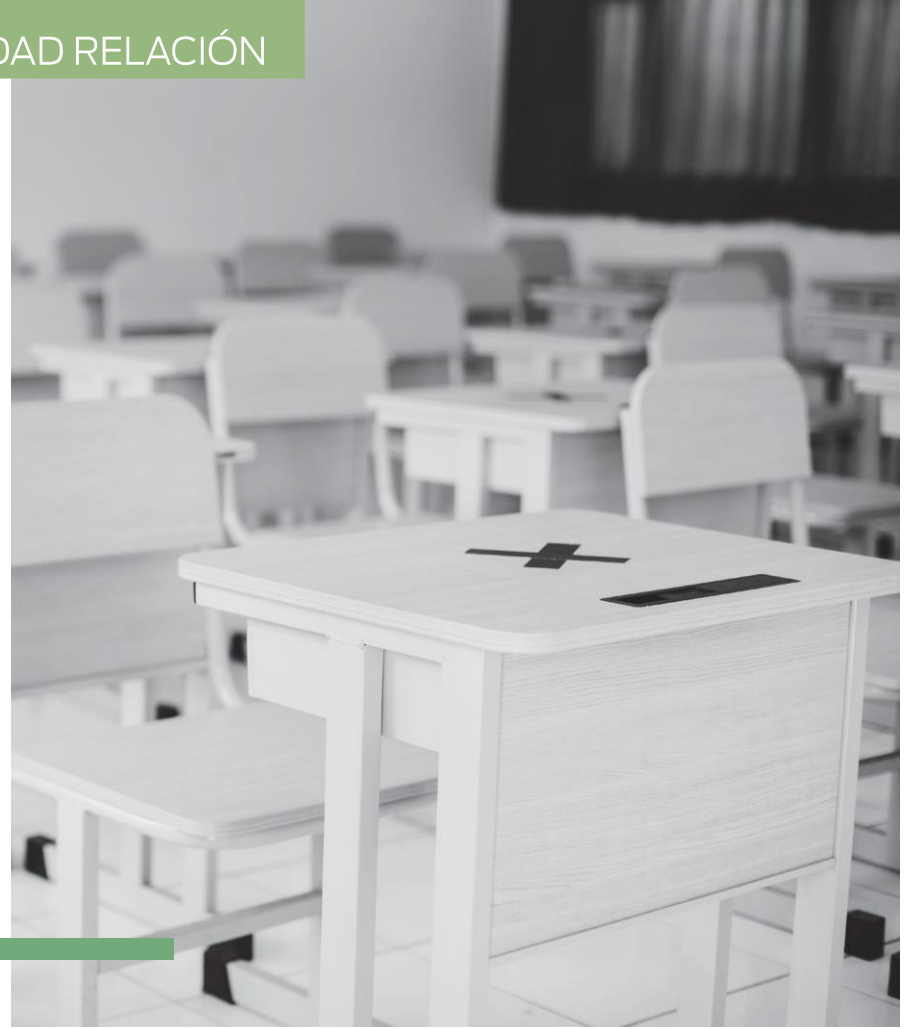
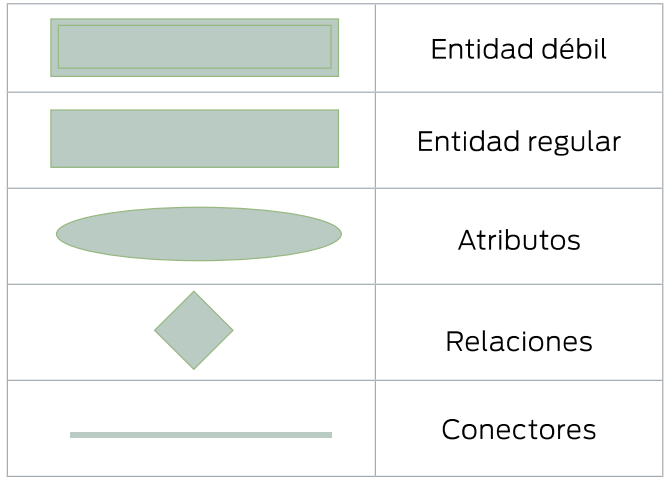


Diagrama entidad-relación (E-R)

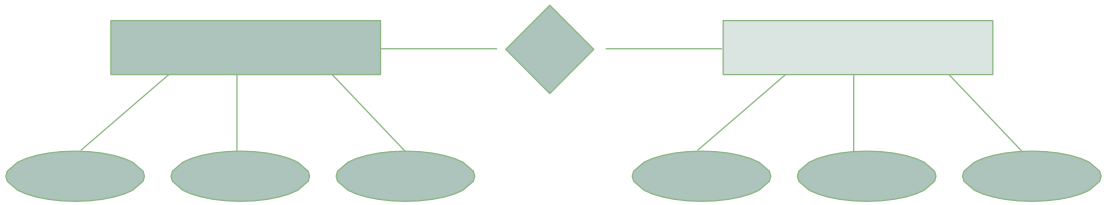
La estructura lógica general de una base de datos se puede demostrar gráficamente mediante un Diagrama E-R el cual tiene los elementos siguientes:



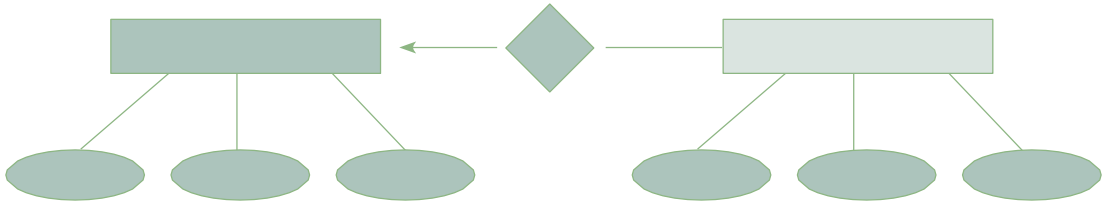
Cada elemento se etiqueta con el nombre que corresponde.

Para diferenciar la cardinalidad en los diagramas, se incorpora a los conectores una flecha según sea esta:

Muchas a muchas

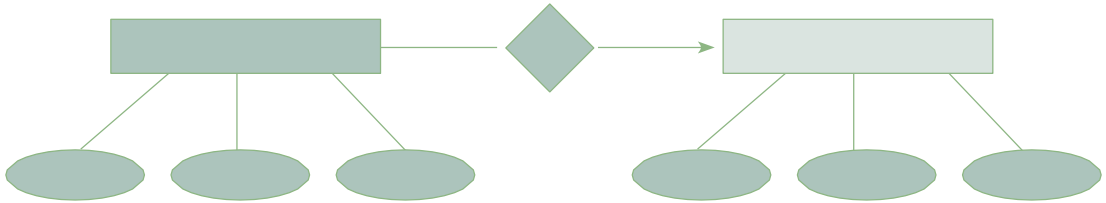


Una a muchas

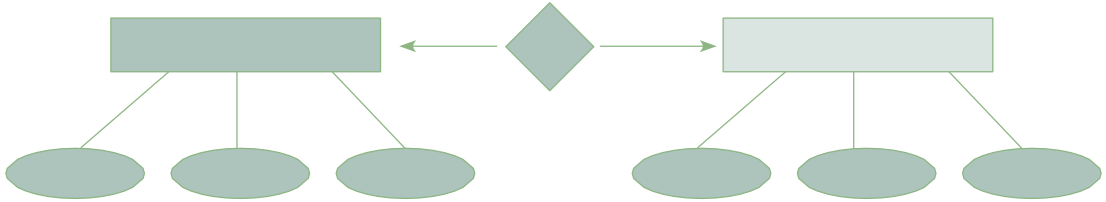


MODELO ENTIDAD RELACIÓN

Muchas a una



Una a una



Dependencia de existencia

La dependencia de existencia es otra clase importante de restricciones.

Implica que la realidad de una entidad depende de la realidad de otra.

Si la vida de la entidad a depende de la realidad de la entidad *b*, entonces se dice que a depende de la realidad de *b*.





Esto implicaría paralelamente que, si se borra *b*, también se eliminará *a*.

En este orden de eventos, a es una entidad subordinada y *b* es una entidad dominante.

En nuestro ejemplo la existencia de una entidad *Alumno* depende de que exista una entidad *Carrera*.

MODELO ENTIDAD RELACIÓN



CLAVE SECUNDARIA
Cualquier clave que no sea la principal.

CLAVE FORÁNEA
Una clave que establece una relación existente entre dos tablas es una referencia a una clave en otra tabla. No tiene que ser una clave única en la tabla en la que reside, pero debe ser única en la tabla desde la que se hace referencia.

CLAVE ÚNICA
Conjunto de campos cuyos valores identifican de forma única cada registro de la tabla. Puede haber varias claves únicas en una tabla determinada, y cada una se denomina “clave candidata”.

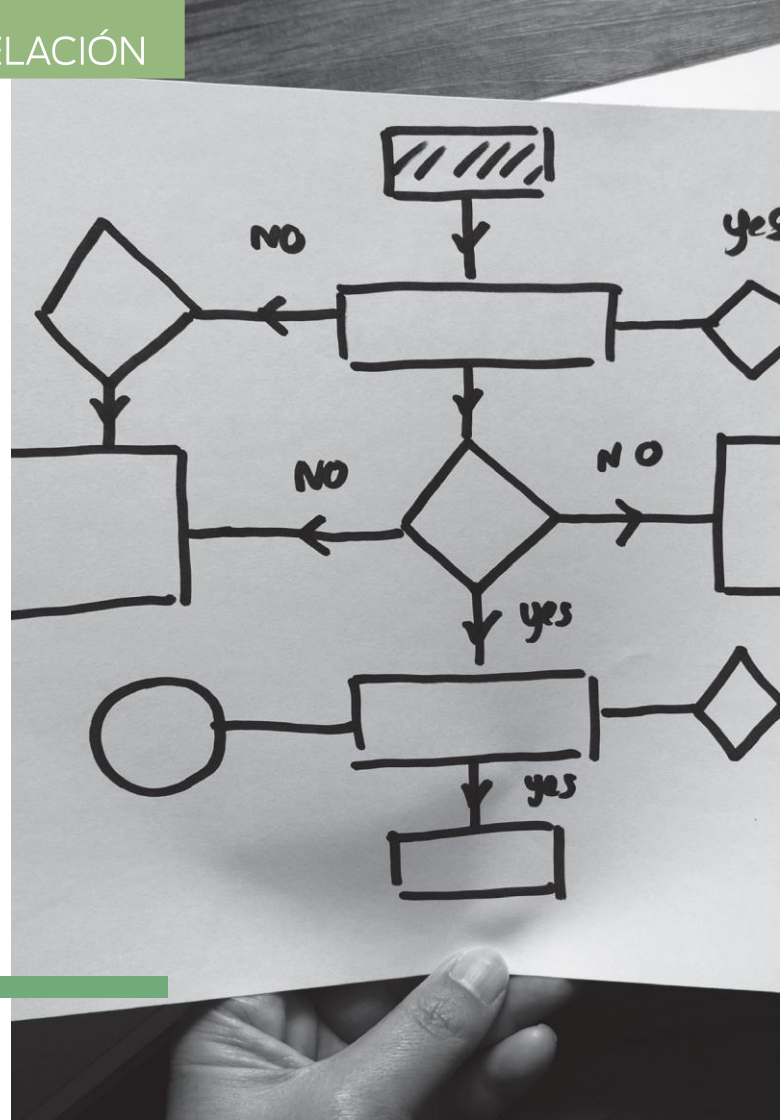
CLAVE PRIMARIA
Una clave única seleccionada de todas las claves candidatas, que define de manera única todos los demás atributos de la tabla, determina los datos que se asociarán con otras tablas. Este es el identificador de la tabla.

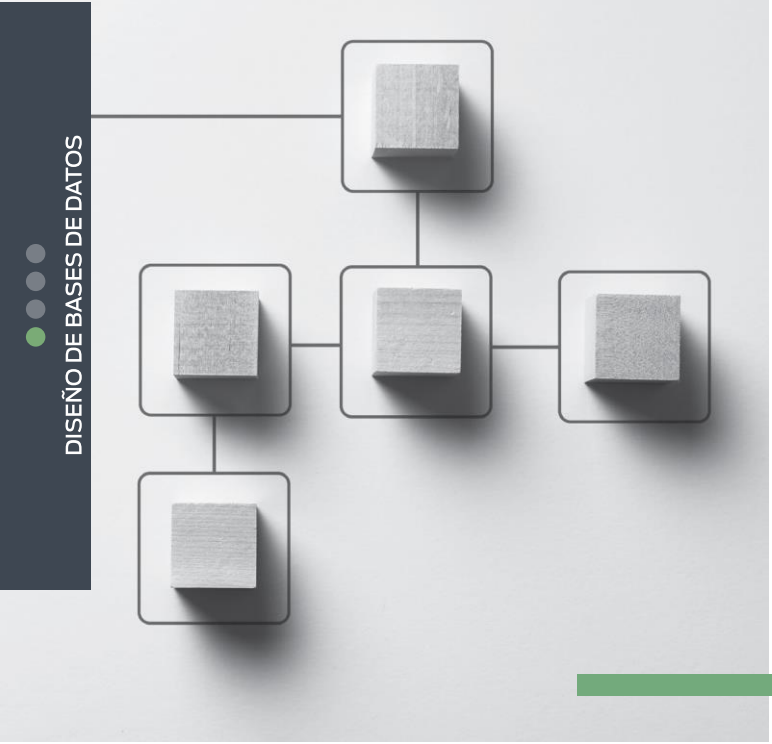
VISTAS
Las tablas virtuales definidas en una o más tablas, tienen la misma estructura de filas y columnas que una tabla, sin embargo, en lugar de almacenar datos, conservan su definición.

Llaves primarias

Desde el punto de vista de una base de datos, las entidades y las relaciones deben ser diferentes entre sí en cuanto a sus atributos. Para hacer estas distinciones, se asigna una superclave a cada grupo de entidades.

Una superclave es un grupo de uno o más atributos que, cuando están vinculados, identifican de forma única un objeto dentro de un grupo de objetos. La entidad Student se puede identificar por su atributo de mapa, por lo que esta es una de sus superclaves.



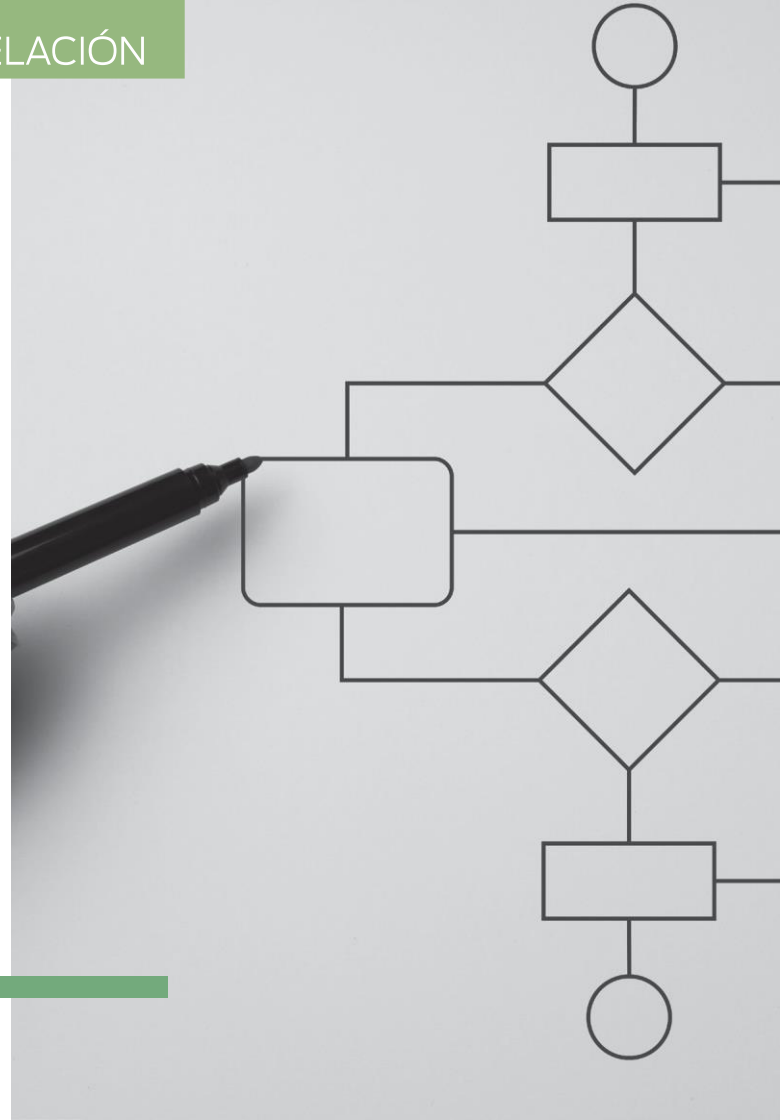


El atributo Nombres por sí solo no es suficiente para identificar a un estudiante, pero cuando se combina con el atributo Tarjeta también puede identificar a un estudiante. Como una mezcla de atributos de tarjeta y otros atributos, ayuda a identificar una entidad.

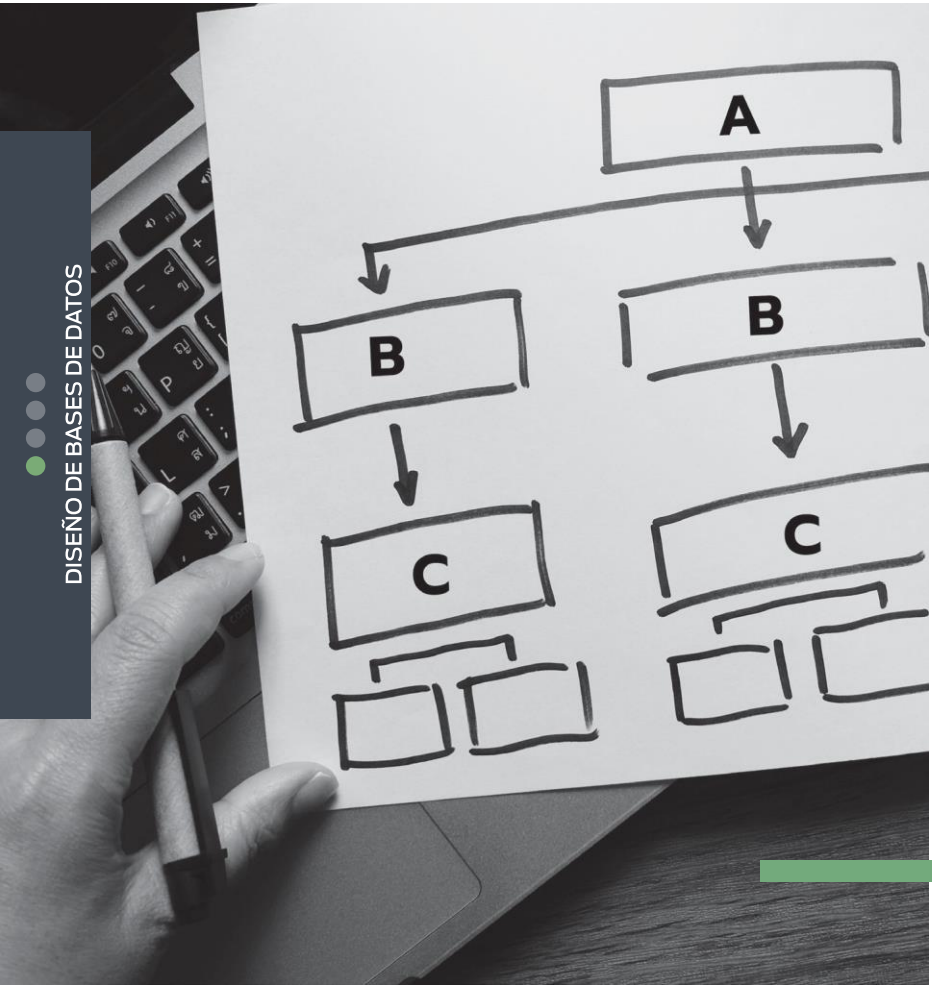
Por esta razón, el concepto de superclaves no es suficiente para el modelado de bases de datos. Lo que quiere en ese caso es la superclave viable más pequeña. Esto se conoce como clave candidata. Puede haber algún conjunto de atributos que se puedan usar como claves candidatas.

El atributo de *nombres* no es suficiente para detectar de forma única a un estudiante, pero si se combina con el atributo ID, también identificaría al estudiante. De esta forma, cualquier mezcla del atributo de la carta con otro de los atributos serviría para detectar una entidad.

Por esta razón, el término superclave no es suficiente para el modelado de bases de datos. Lo que se busca entonces es la superclave más pequeña posible. Esto se conoce como la clave candidata. Existe la posibilidad de que haya varios conjuntos de características que podrían servir como claves candidatas.



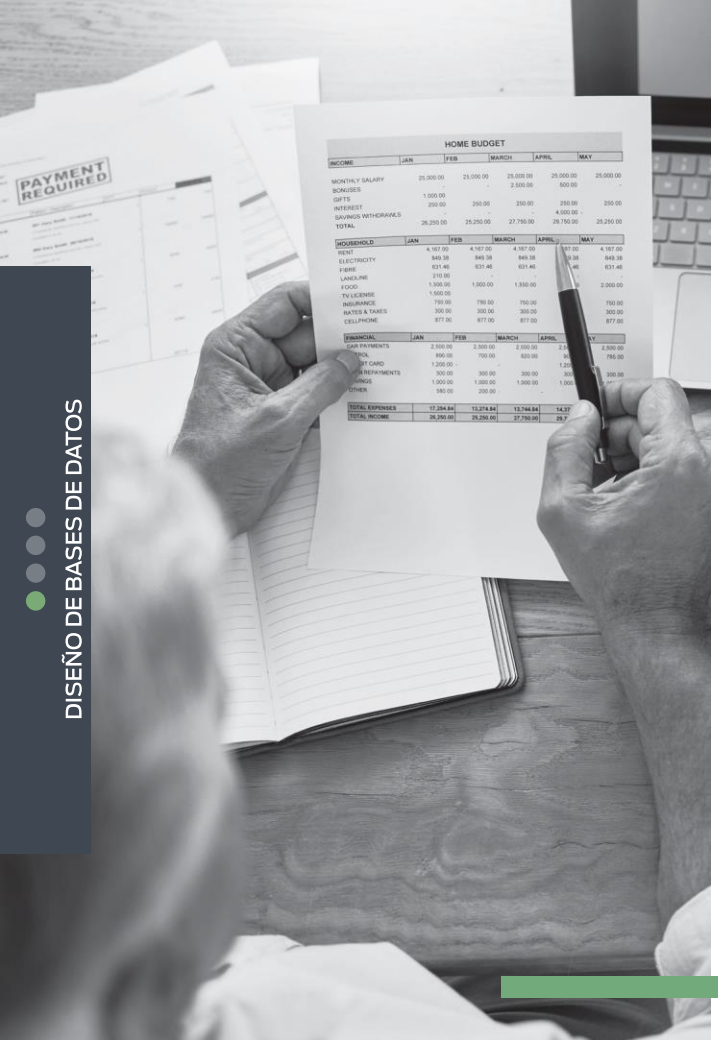
MODELO ENTIDAD RELACIÓN



Un conjunto de relaciones también tiene una clave principal.

Se construyen recuperando los atributos que componen la clave principal del conjunto de entidades que terminaron el grupo de interacciones.

Estos atributos que se derivan de claves de otras entidades se denominan claves foráneas.



NORMALIZACIÓN

La normalización es un criterio para la participación. El propio principio de normalización sugiere que las tablas de la base de datos eliminarán las inconsistencias y la redundancia y minimizarán las ineficiencias.

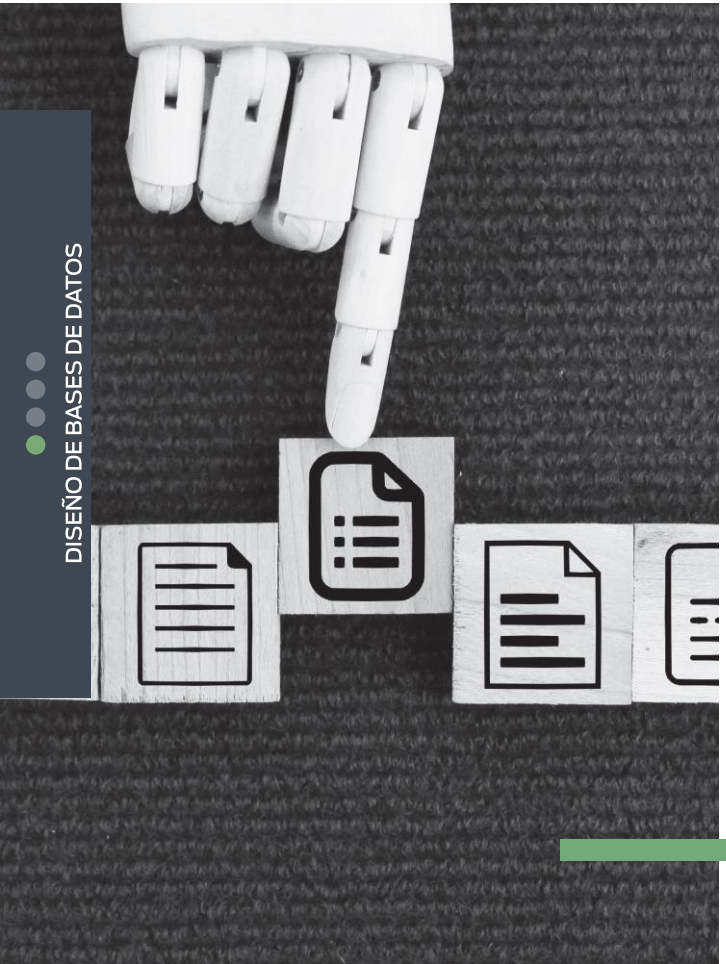
Las bases de datos se describen como inconsistentes cuando sus datos se ingresan de manera ilógica o cuando los datos en una tabla no coinciden con los datos ingresados en otra tabla.

Por ejemplo, si medio personal considera que el municipio de “Monte Azul” está en el área La huasteca Tamaulipeca, de Tamaulipas, y la otra mitad que en la Frontera, y si ambas partes del personal tratan sus datos según esto, los informes de la base de datos acerca de las estadísticas del área La huasteca Tamaulipeca no van a ser significativos.

Una base de datos ineficaz no posibilita aislar los datos precisos que quiere. Una base de datos que almacene todos sus datos en una tabla, obligará a pasar por incontables campos sencillamente para recuperar un dato.



NORMALIZACION



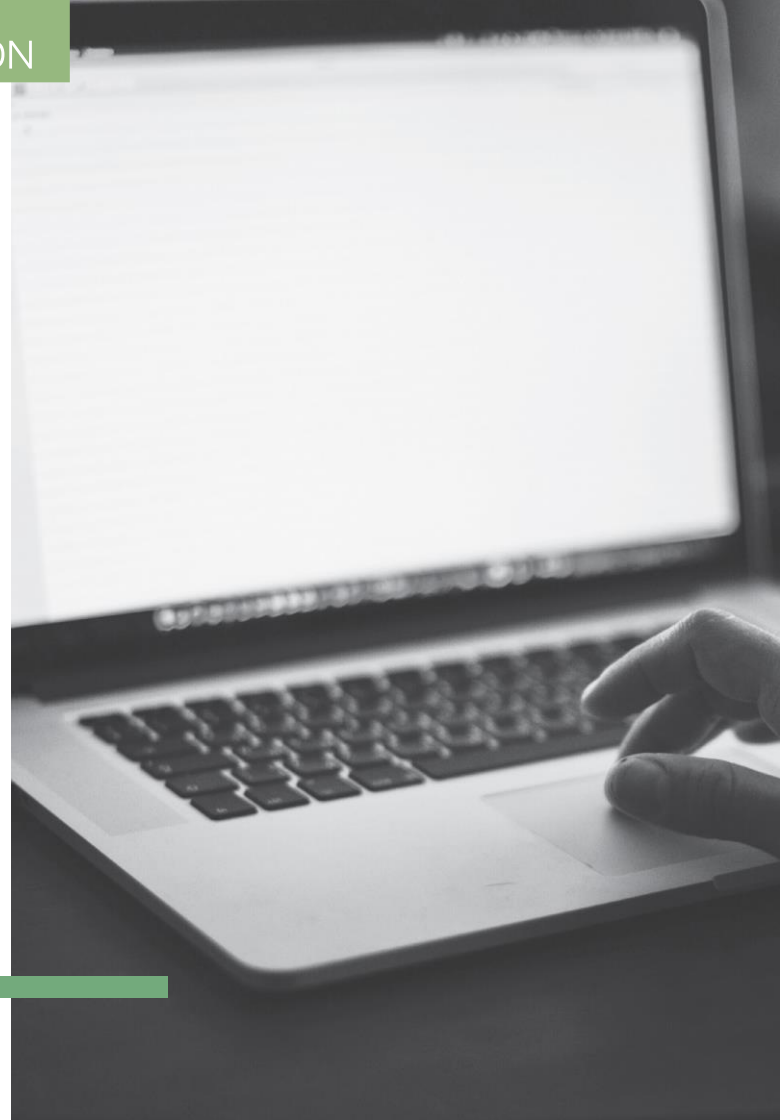
Por otro lado, una base de datos enteramente normalizada almacena cada información en su propia tabla e identifica cada información con su propia clave principal.

Un administrador de base de datos puede hallarse con bases de datos que no fueron normalizadas, por un fundamento u otro.

La carencia de normalización podría ser intencionada o resultado de una falta de experiencia o cuidado del diseñador original de la base de datos.

Sin embargo, si se desea normalizar una base de datos existente, debe hacerse en las primeras etapas del desarrollo de la aplicación. Esto se debe a que depende de la composición de la mesa.

La teoría matemática detrás de la normalización es precisa y compleja, pero las pruebas que se pueden aplicar para asegurar que su diseño tenga sentido son simples.





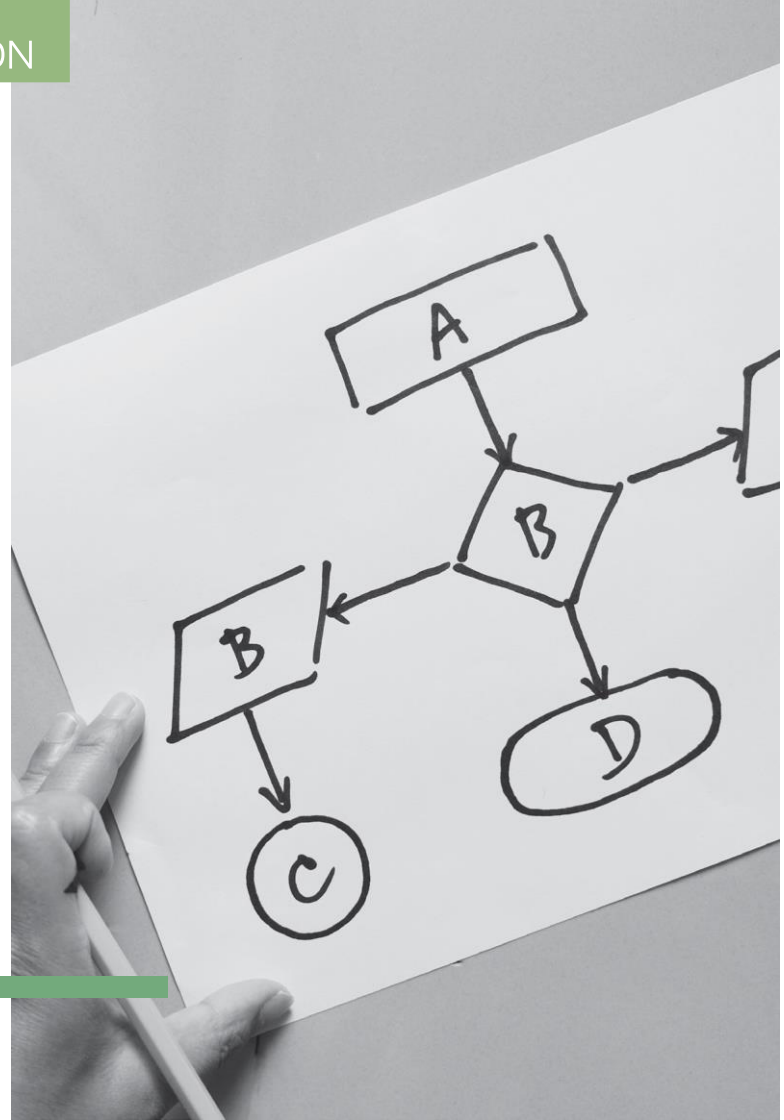
NORMALIZACION

Para ejercer un control de redundancia de datos se recomienda seguir una serie de reglas al momento de crear tablas y establecer relaciones.

La normalización es un instrumento consistente en un conjunto de estas reglas que permanecen en relación unas con otras de una forma directa. Cada regla está basada en la regla que la antecede y va a ser, a la vez, la base de la regla que le precede.

Para llevar a cabo estas reglas, se debe transformar y estructurar la BD hasta que se cumplan las condiciones establecidas para todas ellas. Cada regla es llamada “forma normal”, y si se cumple la primera regla se plantea que la BD está en la “primera norma formal”.

Para llevar a cabo la segunda forma normal es indispensable que se encuentre antes en la primera forma normal, y de esta forma sucesivamente. Una tabla puede estar en primera forma normal y no en segunda forma normal, sin embargo, no al contrario.



VIDEO



Te invitamos a ver el siguiente video:



NORMALIZACION

Primera Forma Normal (1NF)

Es una regla de diseño de bases de datos que establece que cada atributo de una tabla debe contener un único valor atómico, es decir, no debe contener valores repetidos o múltiples valores separados por comas u otros caracteres.

Para cumplir con la 1NF en SQL, es necesario descomponer la tabla original en varias tablas más pequeñas y relacionadas entre sí. Cada tabla debe tener un conjunto de columnas que sean completamente dependientes de la clave primaria de la tabla y que no dependan de otros atributos.

Por ejemplo, supongamos que tenemos una tabla de empleados con los siguientes atributos:

ID	NOMBRE	DEPARTAMENTOS
1	Ana	Ventas, TI
2	Juan	Ventas
3	María	TI

En esta tabla, el atributo “Departamentos” no cumple con la 1NF, ya que algunos empleados pertenecen a más de un departamento y esto se almacena como una lista separada por comas.

Para convertir esta tabla a la 1NF, se debe descomponer en dos tablas: una para los empleados y otra para los departamentos. La tabla de empleados tendría la siguiente estructura:

ID	NOMBRE
1	Ana
2	Juan
3	María

Mientras que la tabla de departamentos tendría la siguiente estructura:

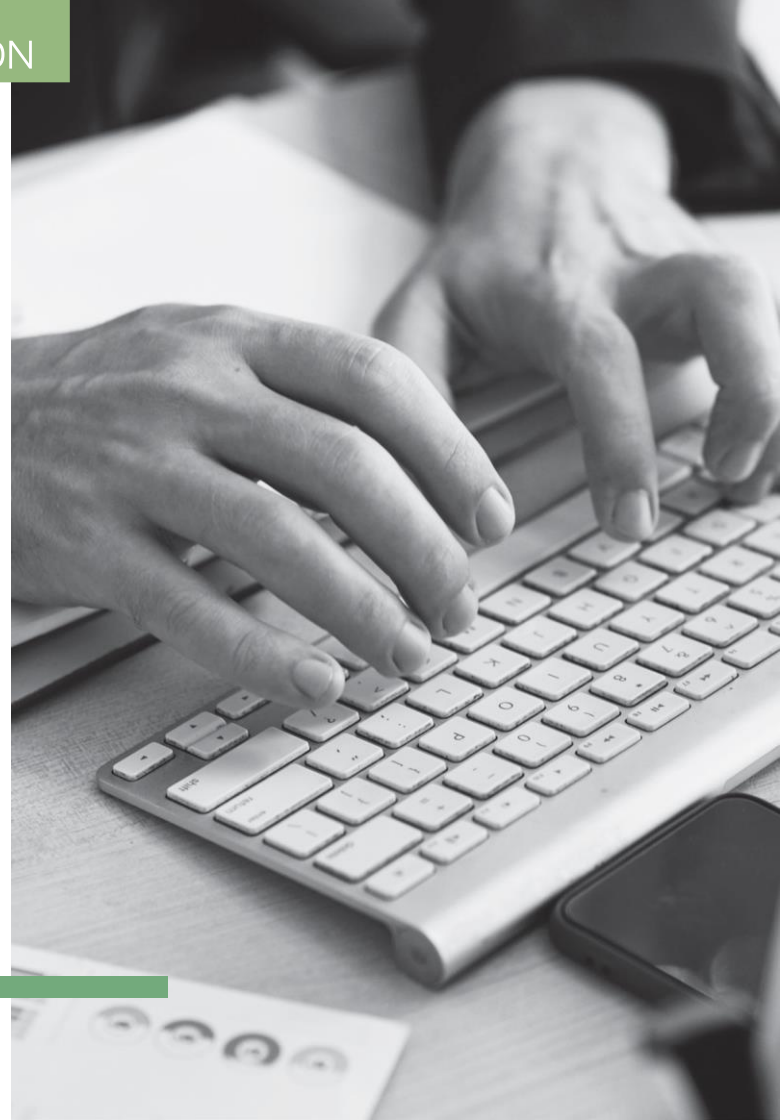
ID	EmpleadoID	Departamento
1	1	Ventas
2	1	IT
3	2	Ventas
4	3	IT

En esta nueva estructura, cada atributo contiene un único valor atómico y la relación entre las tablas se establece mediante la columna EmpleadoID, que es una clave foránea que hace referencia a la tabla de empleados.

Segunda Forma Normal (2NF)

Establece que cada atributo de una tabla debe depender completamente de la clave primaria de la tabla. En otras palabras, cada atributo debe ser funcionalmente dependiente de la clave primaria y no de una combinación de columnas de la tabla.

Para cumplir con la 2NF en SQL, es necesario tener una tabla en la Primera Forma Normal (1NF) y que cada atributo de la tabla dependa completamente de la clave primaria de la tabla.



Por ejemplo, tenemos una tabla de pedidos con los siguientes atributos:

Pedido ID	Fecha	Cliente ID	NombreCliente	Producto ID	NombreProducto	Cantidad	Precio
1	2021-01-01	100	Juan	1	Producto A	2	10.00
2	2021-01-02	200	Ana	2	Producto B	3	5.00
3	2021-01-03	100	Juan	3	Producto C	1	15.00

En esta tabla, el atributo “NombreCliente” y “NombreProducto” no cumplen con la 2NF, ya que no dependen completamente de la clave primaria PedidoID, sino que dependen de ClienteID y ProductoID respectivamente.

Para convertir esta tabla a la 2NF, se debe descomponer en dos tablas: una para los pedidos y otra para los clientes y productos. La tabla de pedidos tendría la siguiente estructura:

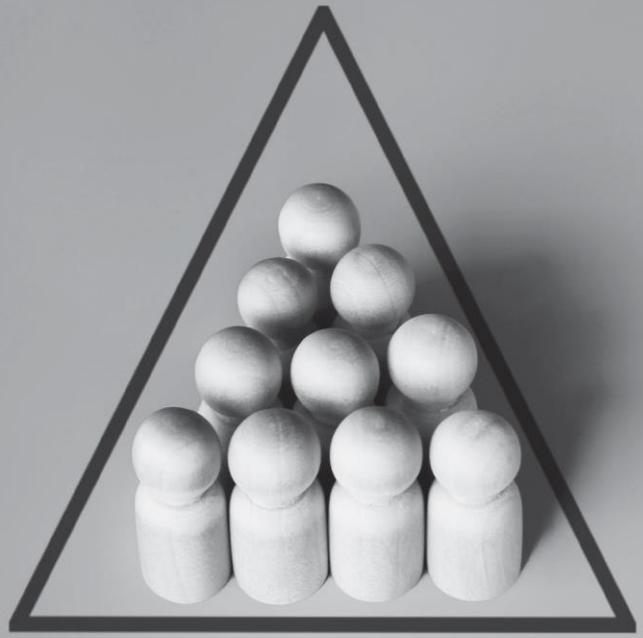
Pedido ID	Fecha	Cliente ID	Producto ID	Cantidad	Precio
1	2021-01-01	100	1	2	10.00
2	2021-01-02	200	2	3	5.00
3	2021-01-03	100	3	1	15.00

Mientras que la tabla de clientes y productos tendría la siguiente estructura:

Cliente ID	NombreCliente
100	Juan
200	Ana

Producto ID	NombreProducto
1	Producto A
2	Producto B
3	Producto C

En esta nueva estructura, cada atributo de la tabla de pedidos depende completamente de la clave primaria PedidoID y no de otras columnas. La relación entre las tablas se establece mediante las claves foráneas ClienteID y ProductoID que hacen referencia a las tablas de clientes y productos, respectivamente.



Tercera Forma Normal (3NF)

Es una regla de diseño de bases de datos que establece que cada atributo de una tabla debe ser funcionalmente dependiente únicamente de la clave primaria de la tabla o de atributos no clave que, a su vez, sean funcionalmente dependientes de la clave primaria.

En otras palabras, la 3NF elimina las dependencias transitivas, es decir, aquellas en las que un atributo depende de otro atributo que no es la clave primaria.

Para entender la 3NF en SQL, veamos la siguiente tabla de ejemplo:

Pedido ID	Fecha	Cliente ID	NombreCliente	Ciudad	País
1	2021-01-01	100	Juan	Madrid	España
2	2021-01-02	200	Ana	Barcelona	España
3	2021-01-03	100	Juan	Buenos Aires	Argentina

En esta tabla, el atributo “NombreCliente” no depende directamente de la clave primaria PedidoID, sino que depende de ClienteID. Por lo tanto, esta tabla no cumple con la 3NF.

Para cumplir con la 3NF, debemos separar la tabla en dos: una tabla para los pedidos y otra para los clientes. La tabla de pedidos quedaría así:

Pedido ID	Fecha	Cliente ID	Ciudad	País
1	2021-01-01	100	Madrid	España
2	2021-01-02	200	Barcelona	España
3	2021-01-03	100	Buenos Aires	Argentina

Mientras que la tabla de clientes quedaría así:

Cliente ID	NombreCliente
100	Juan
200	Ana

De esta forma, se elimina la dependencia transitiva del atributo “NombreCliente” y se garantiza que cada atributo de una tabla dependa únicamente de la clave primaria o de atributos no clave que sean funcionalmente dependientes de la clave primaria.

FORO

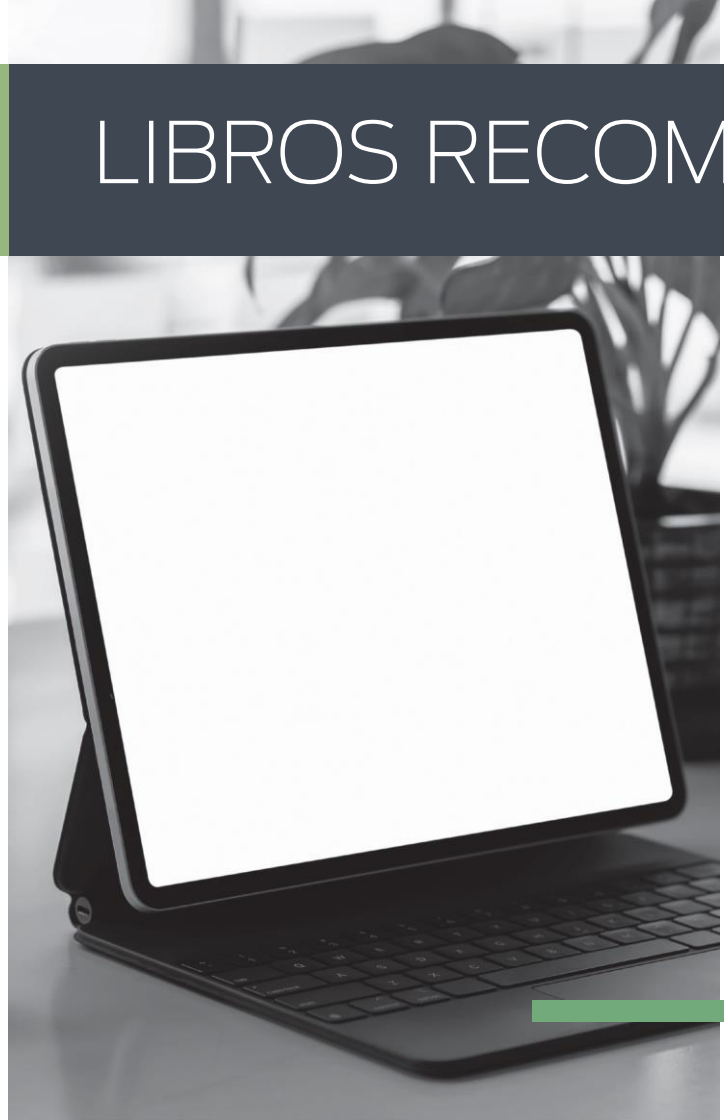
Con los conocimientos que aprendiste en esta unidad, responde la siguiente pregunta:

¿Cómo influye la optimización de la información en el uso de una base de datos en una empresa o negocio?

Presiona el botón para participar en el foro.



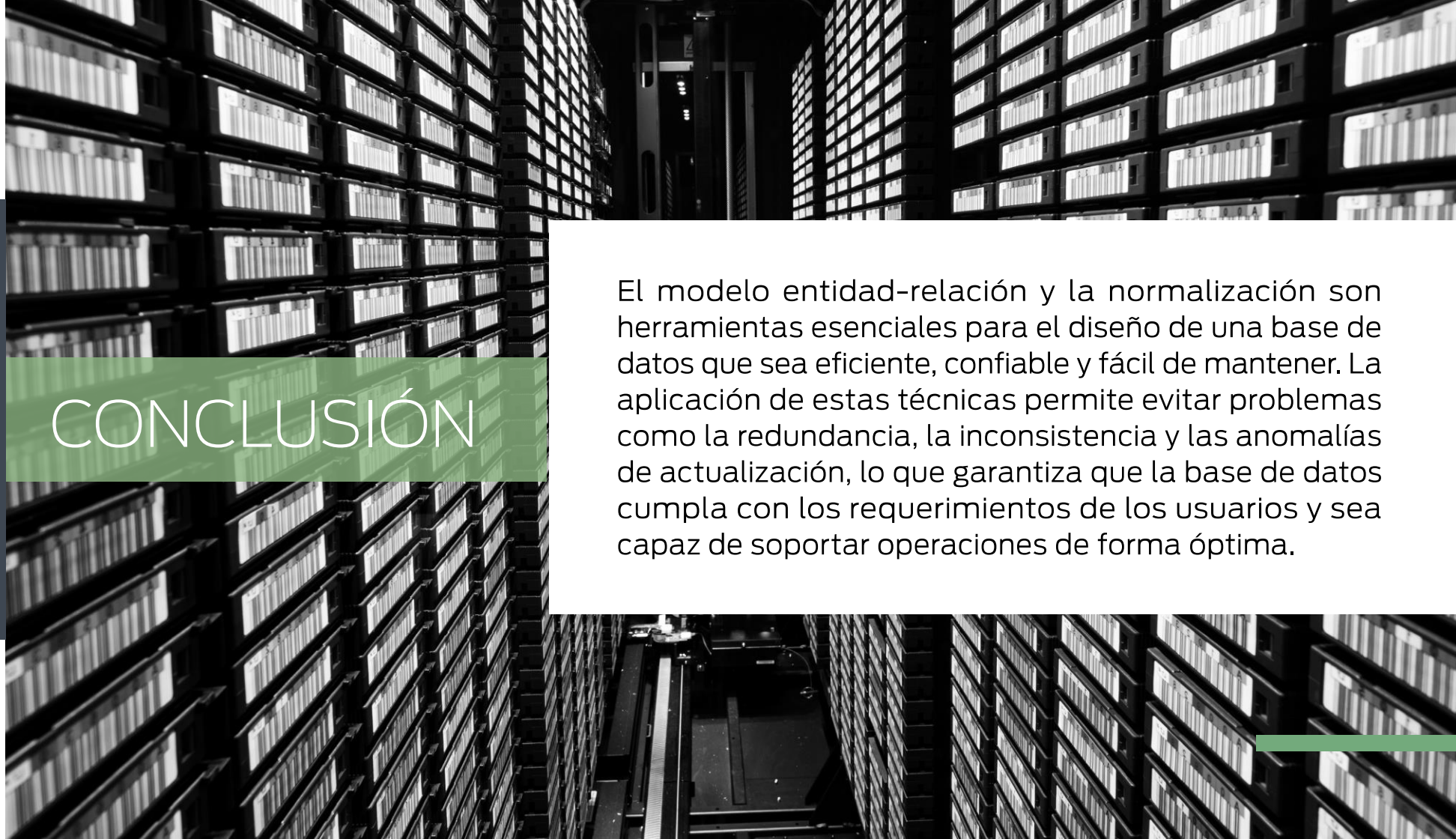
LIBROS RECOMENDADOS



- Benítez,M y Arias, A. (2017). *Curso de Introducción a la Administración de Bases de Datos: 2ª Edición.* De M IT Campus Academy.

- Pulido, E., Escobar, O. y Núñez, J. (2019). *Base de datos.* Editorial Patria.

- Chicano, E. (2014) *Utilización de las bases de datos relacionales en el sistema de gestión y almacenamiento de datos.* IC Editorial.



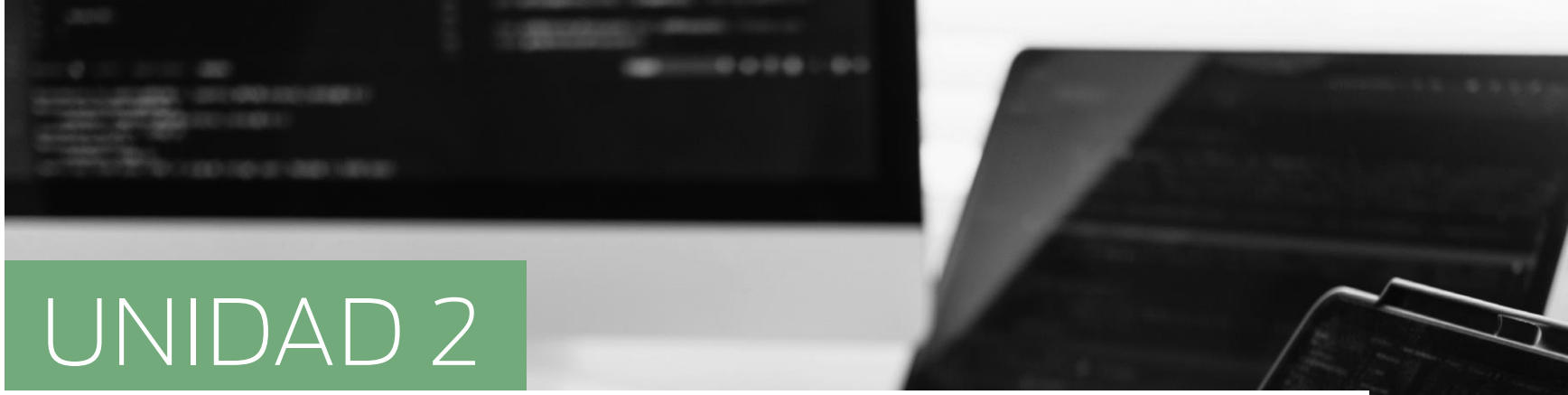
CONCLUSIÓN

El modelo entidad-relación y la normalización son herramientas esenciales para el diseño de una base de datos que sea eficiente, confiable y fácil de mantener. La aplicación de estas técnicas permite evitar problemas como la redundancia, la inconsistencia y las anomalías de actualización, lo que garantiza que la base de datos cumpla con los requerimientos de los usuarios y sea capaz de soportar operaciones de forma óptima.



¡FELICIDADES!

Acabas de concluir la primera unidad de tu curso **Introducción a las Bases de Datos**. Te invitamos a finalizar este esfuerzo realizando el examen parcial correspondiente. Para ello, debes regresar a la pantalla principal y dar clic en Presentar examen.



UNIDAD 2

GESTIÓN DE LA BASE DE DATOS





TEMARIO

2.1 LENGUAJE DE DEFINICIÓN DE DATOS

2.2 LENGUAJE DE MANIPULACIÓN DE DATOS



INTRODUCCIÓN

En esta segunda unidad aprenderás sobre el lenguaje SQL, y a su vez podrás diferenciar el lenguaje de definición de datos del lenguaje de manipulación de datos.

Esta distinción te permitirá comprender de una mejor manera el lenguaje SQL para poder utilizarlo al momento de crear bases de datos.

COMPETENCIAS A DESARROLLAR



- El alumno será capaz de implementar el lenguaje de definición de datos.

- El alumno será capaz de implementar el lenguaje de manipulación de datos.

LENGUAJE DE DEFINICIÓN DE DATOS

La **gestión de la base de datos** es un conjunto de acciones encargadas de controlar los datos.

A medida que aumenta la cantidad de datos, la gestión de la base de datos se convierte en una necesidad para poder controlarla, evitando la pérdida de información y agilizando los procesos de la base de datos.





SQL (Structured Query Language)

SQL es un lenguaje estándar de consulta y manipulación para bases de datos relacionales. Las declaraciones SQL se utilizan para realizar tareas como actualizar o recuperar datos en una base. Algunos sistemas comunes de gestión de bases de datos relacionales que utilizan SQL son: Oracle, Sybase, Microsoft SQL Server, Access, Ingres, etc.

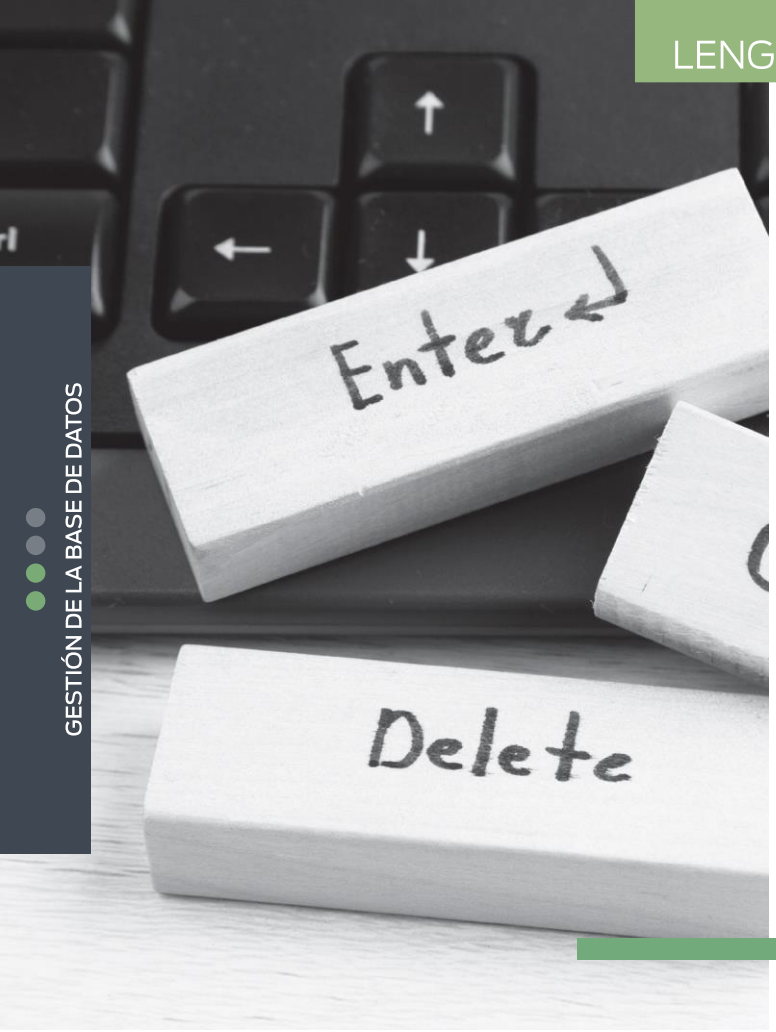
¿Para quién es SQL?

La idea errónea sobre SQL es que es una habilidad de TI y, por lo tanto, solo se aplica a los expertos de la tecnología (no de los negocios).

Los empresarios, gerentes, profesionales de TI e ingenieros pueden avanzar en sus carreras aprendiendo SQL. SQL puede allanar el camino para varias vías y ayudar a las personas a comprender mejor su negocio, a través de los datos que lo impulsan.



LENGUAJE DE DEFINICIÓN DE DATOS



Aunque la mayoría de los sistemas de bases de datos usan SQL, también tienen sus propias extensiones propietarias adicionales, que generalmente solo se usan en su sistema.

Sin embargo, los comandos SQL estándar como “Seleccionar”, “Insertar”, “Actualizar”, “Eliminar”, “Crear” y “Soltar” se pueden usar para lograr casi todo lo que se necesita hacer con una base de datos.

LENGUAJE DE DEFINICIÓN DE DATOS

SQL está regulado por varios organismos de estándares, sin embargo, el más importante es el **ANSI** (Instituto Nacional Estadounidense de Estándares).

Los comandos SQL terminan con un punto y coma (;). Algunas herramientas no requieren lo anterior, pero en la mayoría de los casos, después de escribir una instrucción SQL, se finaliza con un punto y coma.

El lenguaje SQL tiene muchas partes, pero en esta ocasión solo se hablará del **lenguaje de definición de datos (DDL) y del lenguaje de manipulación de datos (DML).**





VIDEO

Te invitamos a ver los siguientes videos:



LENGUAJE DE DEFINICIÓN DE DATOS

DDL

El **lenguaje de definición de datos (DDL)** se utiliza para definir y modificar la estructura de una base de datos relacional. Además, se utiliza para realizar operaciones de definición de datos, tales como:

- CREATE: se utiliza para crear tablas, vistas, índices y otros objetos de bases de datos.
- ALTER: se utiliza para modificar la estructura de una tabla existente, como agregar o eliminar columnas o cambiar el nombre de una tabla.
- DROP: se utiliza para eliminar tablas, vistas, índices y otros objetos de bases de datos.

Además, el DDL también se utiliza para definir restricciones de integridad, como claves primarias, claves foráneas y restricciones de unicidad.

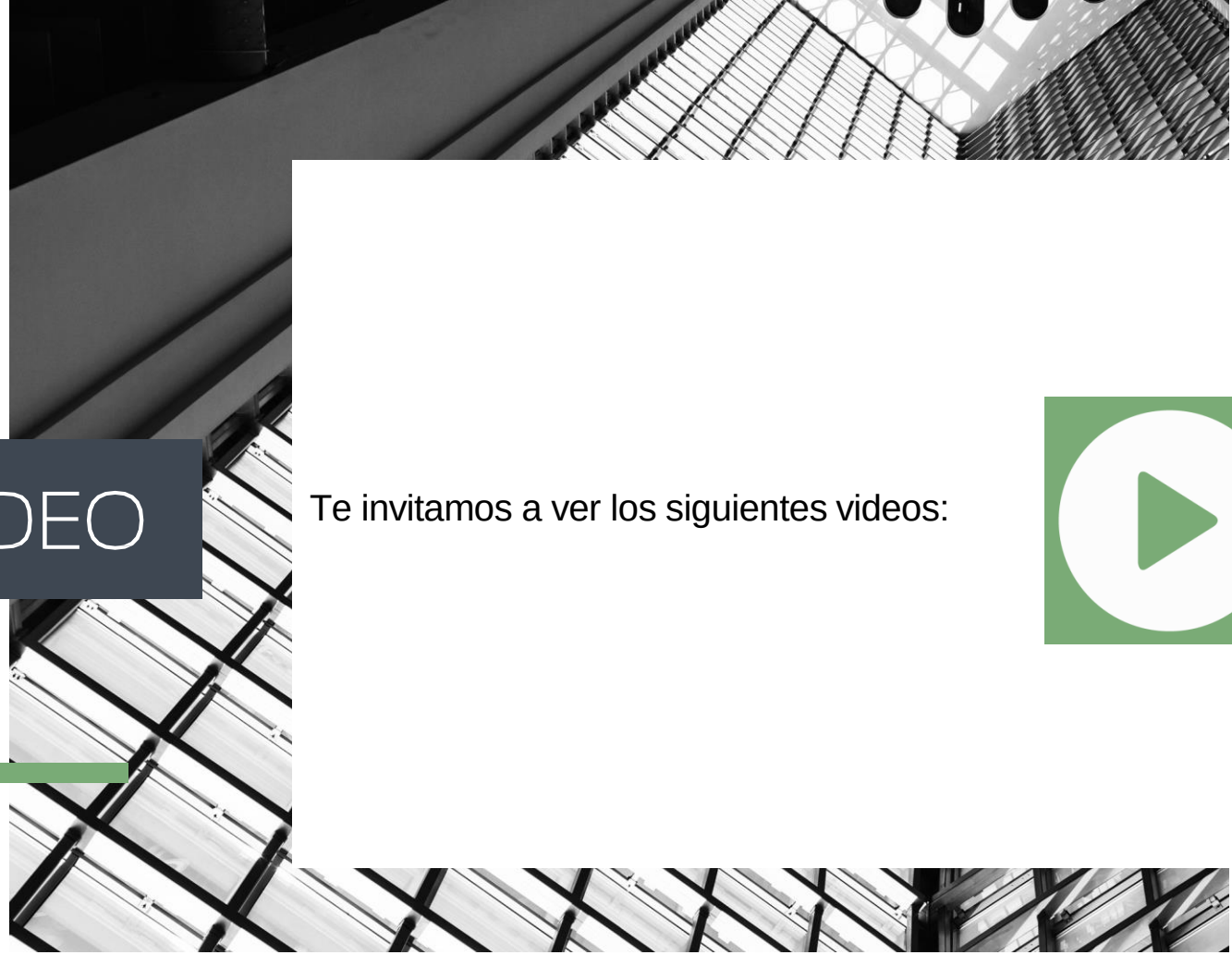
Es importante destacar que, aunque el DDL se utiliza principalmente con bases de datos relacionales, también se puede utilizar para definir y modificar la estructura de otros tipos de bases de datos, como bases de datos NoSQL.



LENGUAJE DE DEFINICIÓN DE DATOS



Se crea la base de datos una vez, se crean sus tablas y después se utilizan. Sus aplicaciones utilizarán DML una y otra vez; sin embargo, generalmente, DDL debe estar completo, antes de que los desarrolladores o usuarios comiencen a utilizar la base de datos.



VIDEO

Te invitamos a ver los siguientes videos:

Ejemplos de sentencias que utiliza el lenguaje de definición de datos:

CREATE

- **CREATE TABLE (crear tabla)** se utiliza para crear una tabla nueva en una base de datos.

Sintaxis

```
CREATE TABLE table_name (
    column1 datatype,
    column2 datatype,
    column3 datatype,
   ....
);
```

Ejemplo:

```
CREATE TABLE Persona (
    IDPersona int,
    Nombre varchar(255),
    Apellido varchar(255),
    Direccion varchar(255)
);
```

- **CREATE TABLE** (crear una copia de una tabla existente)
  La nueva tabla recibe las mismas definiciones de columna. Tiene la posibilidad de elegir cada una de las columnas o columnas concretas.

Si crea una tabla totalmente nueva usando una tabla que existe, la nueva se completará con los valores existentes de la tabla anterior.

Sintaxis

```
CREATE TABLE new_table_name
AS
   SELECT column1, column2,...
   FROM existing_table_name
   WHERE ....;
```

Ejemplo:

```
CREATE TABLE Copia_clientes AS
SELECT Nombre_cliente,
Contacto_cliente
FROM Clientes;
```

DROP

- **DROP TABLE** se utiliza para añadir una tabla existente en una base de datos.

Sintaxis
DROP TABLE table_name;

Ejemplo:
DROP TABLE Vendedores;

ALTER

- **ALTER TABLE (añadir columna)** se usa para modificar la composición de una tabla, sin suprimirla y volver a crearla, como aumentar columnas, borrar columnas o modificar definiciones de columna (por ejemplo, longitud o los valores predeterminados).

Sintaxis

```
ALTER TABLE table_name
ADD column_name datatype;
```

Ejemplo:

```
ALTER TABLE Alumnos
ADD Correo varchar(255);
```

ALTER

- **ALTER TABLE (eliminar columna)** se usa para borrar una columna en una tabla.

Sintaxis

```
ALTER TABLE table_name
DROP COLUMN column_name;
```

Ejemplo:

```
ALTER TABLE Vendedor
DROP COLUMN Correo;
```

PARTITION

- **CREATE PARTITION FUNCTION**

  Permite crear una función en la base de datos actual que asigna las filas de una tabla o índice en particiones, en función de los valores de una columna específica.

Sintaxis

```
CREATE PARTITION FUNCTION partition_function_name (input_parameter_type )
AS RANGE [ LEFT | RIGHT ]
FOR VALUES ( [ boundary_value [ ,...n ] ] )
[ ; ]
```

Ejemplo:

```
CREATE PARTITION FUNCTION pfnTens ( varchar)
AS RANGE RIGHT
FOR VALUES ( 10, 20, 30);
```

Ahora se crea la tabla particionada:

```
CREATE TABLE Empleados (EmpId int, EmpName varchar(50)) on myRangePS2(EmpID);
```

LENGUAJE DE MANIPULACIÓN DE DATOS



DML

Es un subconjunto del lenguaje SQL, que se utiliza para interactuar con bases de datos relacionales.

El DML se utiliza para realizar operaciones de manipulación de datos en una base de datos, tales como:

- SELECT: se utiliza para recuperar datos de una o varias tablas.
- INSERT: se utiliza para agregar nuevos registros a una tabla.
- UPDATE: se utiliza para actualizar los datos existentes en una tabla.
- DELETE: se utiliza para eliminar registros de una tabla.

Además, el DML permite especificar condiciones y restricciones para la manipulación de datos, como la selección de registros que cumplan ciertas condiciones o la actualización de registros que cumplan ciertos criterios.

Es importante destacar que, aunque el DML se utiliza principalmente con bases de datos relacionales, también se puede utilizar para manipular datos en otro tipo de bases de datos, como NoSQL.



Ejemplos de sentencias que utiliza el lenguaje de manipulación de datos:

INSERT

- **INSERT INTO** es utilizado para insertar nuevos registros en una tabla.

Sintaxis

```
INSERT INTO table_name
VALUES (value1, value2, value3, ...);
```

Ejemplo:

```
INSERT INTO Medicina
VALUES ('Paracetamol', 'Omeprazol');
```

INSERT

- **INSERT INTO (datos en columnas)** se utiliza para insertar datos solo en columnas específicas.

Sintaxis

```
INSERT INTO table_name (column1, column2, column3, ...)
VALUES (value1, value2, value3, ...);
```

Ejemplo

```
INSERT INTO Cliente (Nombre_cliente, Ciudad, Teléfono)
VALUES ('Fernando', 'Sinaloa', '3453212354');
```

UPDATE

- **UPDATE** se usa para modificar los registros existentes en una tabla, con la ayuda de WHERE, donde este determina cuántos registros se actualizarán.

Sintaxis

```
UPDATE table_name
SET column1 = value1, column2
= value2, ...
WHERE condition;
```

Ejemplo:

```
UPDATE Alumno
SET Contacto_alumno = 'Alfredo
Castro', Ciudad= 'Tampico'
WHERE ID_alumno = 3;
```

DELETE

- **DELETE (registro en particular)** se usa para borrar registros existentes en una tabla, con la ayuda de WHERE se determina qué registro se eliminará de la tabla.

Sintaxis

```
DELETE FROM table_name
WHERE condition;
```

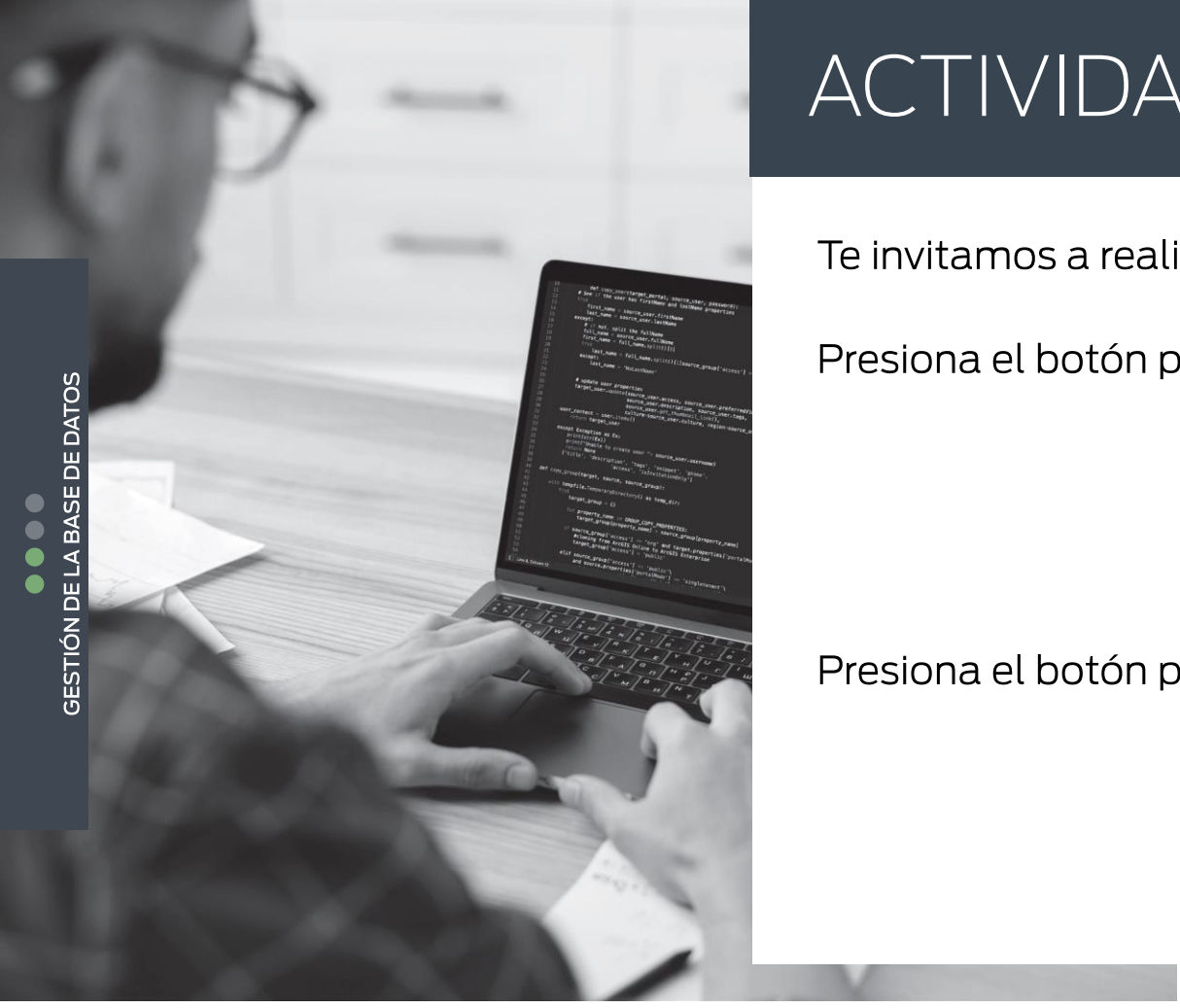
Ejemplo:

```
DELETE FROM Pacientes WHERE
Nombre_paciente='Abiran
Castillo';
```

DELETE

- **DELETE (todos los registros de una tabla)** se usa para borrar todos los registros existentes en una tabla.

Sintaxis	Ejemplo:
DELETE FROM table_name	DELETE FROM Ventas;



GESTIÓN DE LA BASE DE DATOS

ACTIVIDAD INTEGRADORA 1

Te invitamos a realizar la siguiente actividad.

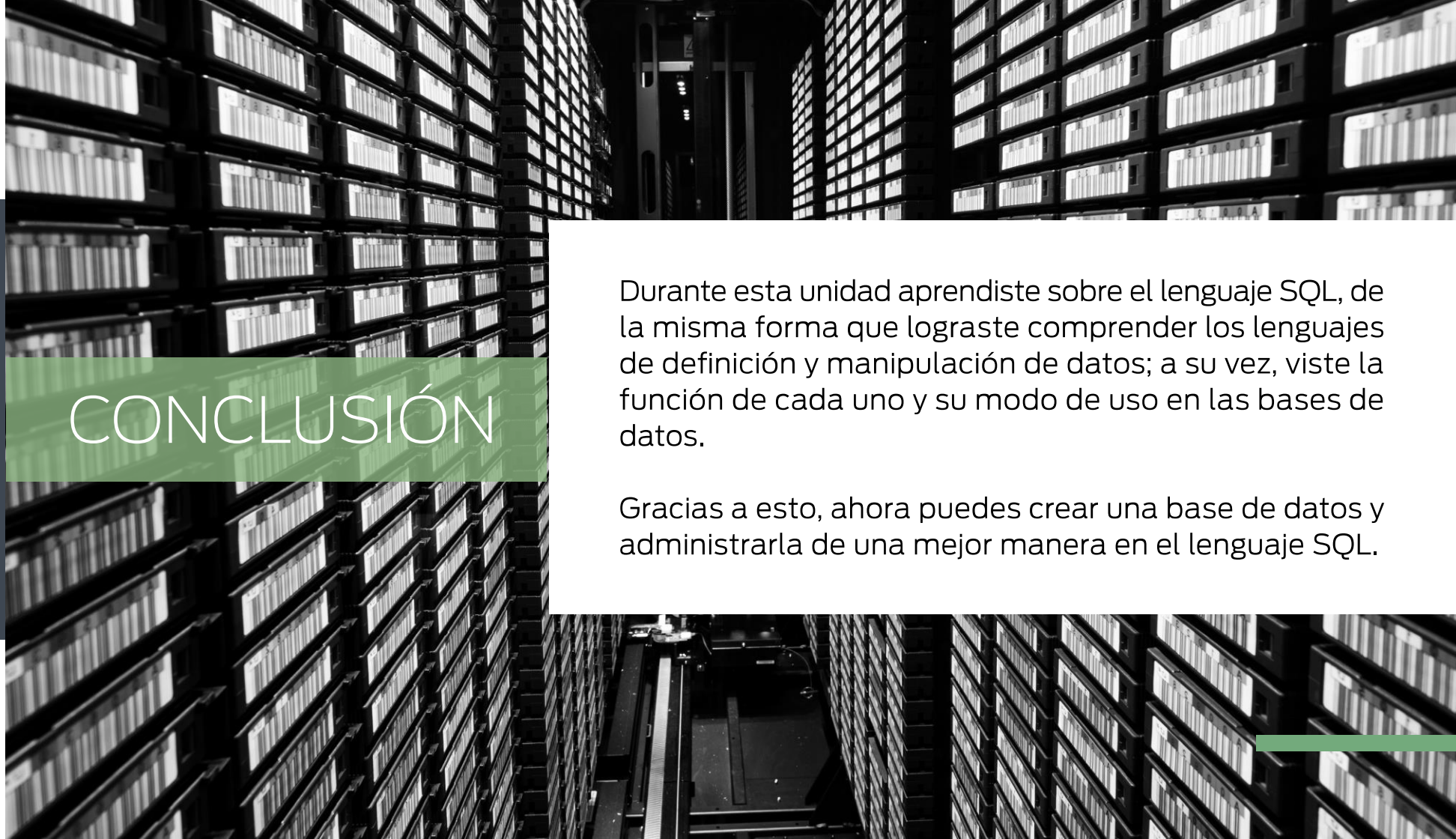
Presiona el botón para descargar la actividad.

Presiona el botón para entregar la actividad.

LIBROS RECOMENDADOS



- Cunningham, L. (2008). *The SQL Starter Series Book 1 -DML An Introduction to the SQL Data Manipulation Languaje.*



CONCLUSIÓN

Durante esta unidad aprendiste sobre el lenguaje SQL, de la misma forma que lograste comprender los lenguajes de definición y manipulación de datos; a su vez, viste la función de cada uno y su modo de uso en las bases de datos.

Gracias a esto, ahora puedes crear una base de datos y administrarla de una mejor manera en el lenguaje SQL.



¡FELICIDADES!

Acabas de concluir la segunda unidad de tu curso **Introducción a las Bases de Datos**. Te invitamos a finalizar este esfuerzo realizando el examen parcial correspondiente. Para ello, debes regresar a la pantalla principal y dar clic en Presentar examen.



UNIDAD 3

CONSULTAS ESTÁTICAS





TEMARIO

3.1 AGREGACIONES Y AGRUPACIONES

3.2 UNIONES Y SUBCONSULTAS



INTRODUCCIÓN

En esta tercera unidad aprenderás a utilizar las funciones de agregación y agrupación, y conocerás las diferencias de cada una mediante ejemplos.

A su vez, conocerás las uniones y subconsultas, su sintaxis y modo de uso en las bases de datos SQL.

COMPETENCIAS A DESARROLLAR



- El alumno será capaz de implementar agregaciones y agrupaciones a las bases de datos.

- El alumno será capaz de implementar el lenguaje de manipulación de datos.

AGREGACIONES Y AGRUPACIONES

Consultas estáticas

Las consultas estáticas son aquellas que están escritas en el código de la aplicación y no cambian en tiempo de ejecución. Estas consultas son útiles cuando se sabe de antemano qué información se necesita obtener de la base de datos y no se necesita ejecutar consultas dinámicas basadas en la entrada del usuario o datos en tiempo real.

Para realizar consultas estáticas a la base de datos, puedes utilizar un lenguaje de consulta como SQL.

Por ejemplo, si tienes una tabla de “usuarios” con los campos “id”, “nombre”, “apellido” y “correo electrónico”, puedes realizar una consulta estática para obtener el nombre y apellido de todos los usuarios:

```
SELECT nombre, apellido FROM usuarios;
```

Funciones de agregación

Las funciones de agregación en bases de datos son funciones que se utilizan para realizar cálculos sobre un conjunto de valores y devolver un único valor de resultado. Estas funciones son útiles para obtener información resumida de grandes conjuntos de datos.

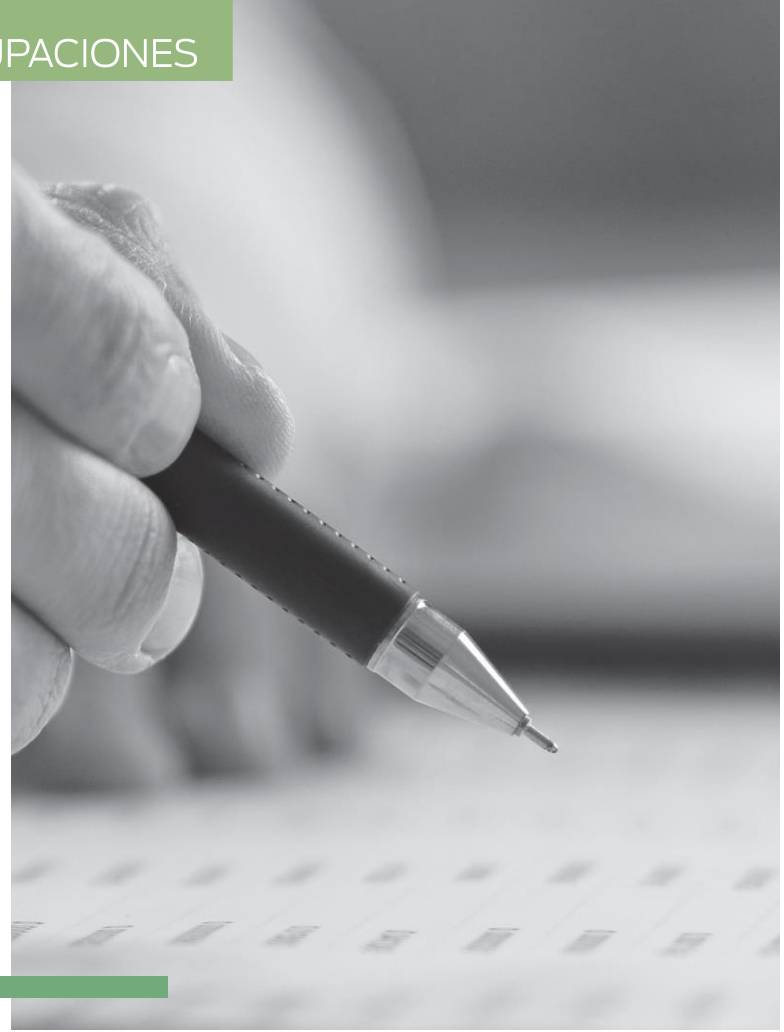
Algunas de las funciones de agregación más comunes son:

COUNT
SUM
AVG
MAX
MIN

Se utiliza una función agregada cuando se desea calcular la información de resumen de una o varias tablas.

- **COUNT** cuenta el número de registros en una tabla o el número de valores no nulos en una columna.

Algunas veces, solo deseamos saber cuántos registros existen en una tabla sin mostrar realmente todo el contenido de estos registros. Esto se puede lograr mediante el uso de esta función.



Una gran virtud de los sistemas basados en SQL sobre las resoluciones de almacenamiento de datos NoSQL es lo intuitivas que son en la agrupación y la agregación de la primera categoría.

Ejemplo:

Materias

ID_materia	Materia	Carrera	Grado
1	Base de datos	ISC	1
2	Programación	ISC	
3	Ética	IM	
4	Matemáticas	Contador	2
6	Aplicaciones Móviles	ISC	8

A simple vista vemos que existen 6 filas en la tabla, pero en realidad no lo es, ya que puede pasar, como en este ejemplo, que se haya eliminado un ID y nadie lo note, por lo que en este caso necesitamos de COUNT.

Sintaxis

SELECT COUNT(*) FROM Materias;

La salida devuelta será un solo registro con un solo campo con el valor 5. La función COUNT tomó todos los registros ya que añadimos un (*), el cual significa que deberá tomar todos los registros existentes. Así logramos el objetivo de contar todos los registros.

¿Qué pasaría si en lugar de solicitar un registro completo para contar, especificamos una columna? ¿Y si la columna tuviera dentro de ella valores nulos?

Veamos esta situación contando el campo Grado de la tabla:

Materias

ID_materia	Materia	Carrera	Grado
1	Base de datos	ISC	1
2	Programación	ISC	
3	Éticaw	IM	
4	Matemáticas	Contador	2
6	Aplicaciones Móviles	ISC	8

Sintaxis

SELECT COUNT(Grado) FROM Materias;

Si el resultado devuelto de esta consulta fue 3, es correcto.

De las cinco filas, dos registros contienen valor nulo (0) en el campo Grado, por lo cual los ignora y cuenta los que están realmente.

Utilizar DISTINCT con COUNT

La cláusula DISTINCT nos permite mostrar solo valores no duplicados de la entrada especificada y se usa comúnmente junto con COUNT.

Sintaxis

De acuerdo a la tabla de ejemplo:

SELECT DISTINCT Carrera FROM Materias;

Materias

ID_materia	Materia	Carrera	Grado
1	Base de datos	ISC	1
2	Programación	ISC	
3	Ética	IM	
4	Matemáticas	Contador	2
6	Aplicaciones Móviles	ISC	8

El resultado devuelto será:

Carrera
IM
Contador

Una forma común de uso de DISTINCT es combinarlo con la función COUNT para generar el número de valores únicos que tenemos en la tabla.

El uso de DISTINCT en el campo Grado tiene un resultado ligeramente diferente de lo que podríamos esperar.

Sintaxis

SELECT COUNT (DISTINCT Grado) FROM Materias;

Resultado:

Grado
1
(null)
(null)
2
8

De hecho, obtenemos cinco filas en nuestro resultado, incluidos los valores nulos porque para la cláusula DISTINCT, es un valor separado único.
Combinar esto con COUNT elimina el significado de las filas nulas y nos da el valor 3.

- **AVG** calcula el promedio de los valores de una columna numérica.

Veamos la siguiente tabla para utilizar esta función.

Libros

ID_libro	Nombre_libro	Autor_libro	Año_ publicado
1	Programación	Francisco Pérez	2013
2	Base de datos	Manuel Cruz	2011
3	Algoritmos	Pedro Paz	1988
4	Variables	Ernesto Villa	2018

En esta consulta queremos averiguar el valor anual medio de todos los libros publicados que se muestran en la tabla anterior.

Por lo cual utilizaremos la función AVG de la siguiente manera:

Sintaxis

```
SELECT AVG(Año_publicado)
  FROM Libros;
```

Resultado

avg
2007.5

Si bien el valor promedio se calculó con exactitud, se puede decir que un valor como este para especificar el año no es eficaz; lo que en verdad deseamos es un valor de año legible que parezca un año real, especialmente un valor entero. Por consiguiente, vamos a convertir este valor promedio en un número entero.

Conversión de valores a enteros

```
SELECT CAST(AVG(Año_publicado) AS INTEGER)
FROM Libros;
```

La conversión de tipos de datos por medio de CAST solo funciona con tipos de datos compatibles, como números y números enteros. Si intentas convertir un varchar en un entero, el Sistema de administración de bases de datos (DBMS) que utilices, arrojará un mensaje de error muy parecido a este (en caso de seleccionar la columna Nombre_libro de la tabla):

```
testdb=# SELECT CAST(Nombre_libro AS INTEGER)
FROM Libros;
ERROR: invalid input syntax for integer: “Programacion“
```

O incluso otro DBMS podrá arrojar un 0 como resultado.

- **SUM:** suma los valores de una columna numérica.

Con base a la siguiente tabla, queremos encontrar el resultado de la suma de los valores de los años de la tabla Libros, para esto utilizaremos en la consulta la función SUM de la siguiente manera:

Libros

ID_libro	Nombre_libro	Autor_libro	Año_ publicado
1	Programación	Francisco Pérez	2013
2	Base de datos	Manuel Cruz	2011
3	Algoritmos	Pedro Paz	1988
4	Variables	Ernesto Villa	2018

Sintaxis

```
SELECT SUM(Año_publicado)
FROM Libros;
```

Resultado

```
sum
8030
```

De igual forma como ya lo mencionamos, si utilizas esta función en un campo varchar, el resultado puede ser un error o un 0, dependiendo del DBMS que utilices.



- **MAX:** devuelve el valor máximo de una columna.
- **MIN:** devuelve el valor mínimo de una columna.

MAX y MIN se denominan funciones extremas porque esencialmente encuentran los valores extremos de un conjunto de valores en una columna. Su uso más intuitivo es con datos numéricos, pero estas funciones también se pueden aplicar en otro tipo de bases de datos.

Estas funciones son bastante fáciles de entender y usar.

MIN

Esta función examina un conjunto dado de filas y encuentra el valor mínimo de la columna dada como argumento.

Por ejemplo, en la siguiente tabla, queremos saber el año de publicación del libro más antiguo. Estudiando el problema, vemos que si aplicamos la función agregada MIN al campo año_publicado de nuestra tabla, obtendremos el resultado deseado.



Libros

ID_libro	Nombre_libro	Autor_libro	Año_ publicado
1	Programación	Francisco Pérez	2013
2	Base de datos	Manuel Cruz	2011
3	Algoritmos	Pedro Paz	1988
4	Variables	Ernesto Villa	2018

Sintaxis

```
SELECT MIN(Año_publicado)
FROM Libros;
```

Resultado

```
min
1988
```

MAX

La función MAX es similar a la función MIN, pero su resultado sería el último año en el que se publicó uno de los libros de nuestra tabla. Ejemplo

Sintaxis	Resultado
SELECT MAX(Año_publicado) FROM Libros;	max 2018

De igual forma, se pueden combinar las dos funciones MIN y MAX en una sola consulta:

Sintaxis

```
SELECT MAX(Año_publicado),
MIN(Año_publicado)
FROM Libros;
```

Resultado

max	min
2018	1988

Como se mencionó anteriormente, estas funciones no se limitan a tipos de valores numéricos.

Así que combinemos la búsqueda del valor MAX del año de publicación de un libro con el valor MIN del nombre de un libro:

Libros

ID_libro	Nombre_libro	Autor_libro	Año_ publicado
1	Programación	Francisco Pérez	2013
2	Base de datos	Manuel Cruz	2011
3	Algoritmos	Pedro Paz	1988
4	Variables	Ernesto Villa	2018

Sintaxis

```
SELECT MAX(Año_publicado),
MIN(Nombre_libro)
FROM Libros;
```

Resultado

max	min
2018	Algoritmos

El idioma MIN encontrado fue Algoritmos ya que es el primero alfabéticamente.



AGREGACIONES Y AGRUPACIONES

Sin embargo, debemos tener cuidado al leer este resultado. A simple vista da una impresión engañosa de que 2018 corresponde a Algoritmos, lo cual no es el caso.

Nuestra consulta simplemente nos brinda los valores extremos de estos dos campos en nuestra tabla, sean o no del mismo registro.

Agrupaciones

Las agrupaciones en bases de datos se utilizan para agrupar filas de una tabla según el valor de una o varias columnas, y realizar cálculos de agregación sobre cada grupo. Esto es útil cuando se quiere obtener información resumida de una tabla, organizada por categorías.

- **GROUP BY**

La cláusula GROUP BY de una consulta SELECT se utiliza para agrupar registros según sus valores de campo. Esta cláusula se coloca después del condicional WHERE.

En nuestra tabla de ejemplo queremos agrupar datos por nombre de libros.

Libros

ID_libro	Nombre_libro	Autor_libro	Año_ publicado
1	Programación	Francisco Pérez	2013
2	Base de datos	Manuel Cruz	2011
3	Algoritmos	Pedro Paz	1988
4	Variables	Ernesto Villa	2018

Sintaxis

```
SELECT  Nombre_libro  FROM
Libros
WHERE Nombre_libro IS NOT
NULL
GROUP BY Nombre_libro;
```

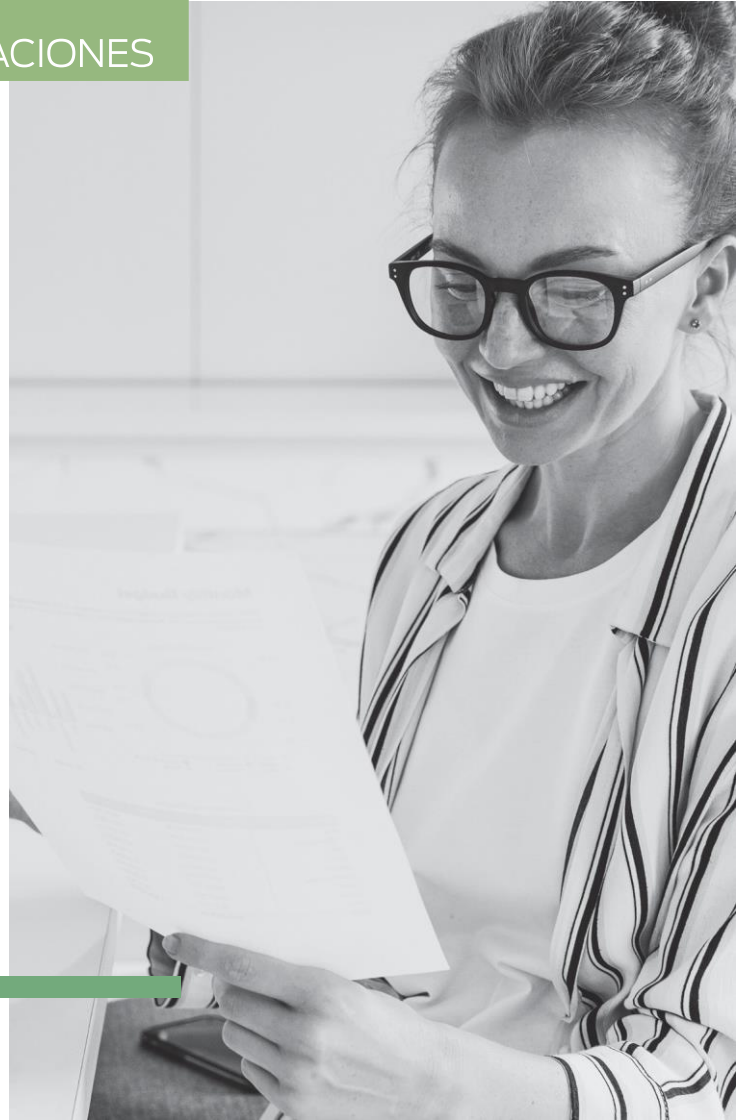
Resultado

Nombre_libro
Algoritmos
Bases de datos
Programación
Variables

La regla a tener en cuenta aquí es que las columnas enumeradas en la cláusula SELECT deben estar presentes en la cláusula GROUP BY. Con eso:

1. No puede agrupar por una columna que no está en la lista SELECCIONAR.

2. Todas las columnas que están presentes en la lista SELECT deben especificarse en la cláusula de agrupación.



Funciones de agrupación y agregación

Una forma muy útil de usar la agrupación es combinar operaciones con funciones de agregación.

Supongamos que queremos contar cuántos autores publicaron sus libros en el mismo año en nuestra tabla.

Esto se puede lograr combinando la cláusula GROUP BY con la función agregada COUNT, como se muestra a continuación:

Sintaxis

```
SELECT Autor_libro,
COUNT(*)
FROM Libros
GROUP BY Autor_libro;
```

Resultado

Autor_libro	COUNT(*)
Ernesto Villa	1
Francisco Pérez	1
Manuel Cruz	1
Pedro Paz	1



El resultado es intuitivo, pero la consulta en sí es interesante.

Hay que tener cuenta que la cláusula GROUP BY consistía solo en los nombres de los autores. La función agregada es el resultado del agrupamiento de las columnas agrupadas.

Veamos otro ejemplo de combinar GROUP BY con múltiples funciones agregadas esta vez.

Supongamos que deseamos averiguar cuántos libros se publicaron en el mismo año, y cuáles son los primeros en orden alfabético.

Inmediatamente podemos ver que aquí se necesita un GROUP BY con Año_publicado, junto con un par de funciones agregadas diferentes.

Sintaxis

```
SELECT Año_publicado,
MIN(Nombre_libro),
COUNT(*)
FROM Libros
GROUP BY Año_publicado;
```

Resultado

Año_publicado	MIN(Nombre_libro)	COUNT(*)
1988	Algoritmos	1
2011	Bases de datos	1
2013	Programación	1
2018	Variables	1

- **HAVING:** se utiliza para filtrar grupos según una condición de agregación.

Al igual que una cláusula WHERE, que coloca condiciones en los campos de una consulta, la cláusula HAVING coloca condiciones en los grupos creados por GROUP BY. Debe colocarse inmediatamente después de GROUP BY pero antes de la cláusula ORDER BY. Ejemplo:

Libros

ID_libro	Nombre_libro	Autor_libro	Año_ publicado
1	Programación	Francisco Pérez	2013
2	Base de datos	Manuel Cruz	2011
3	Algoritmos	Pedro Paz	1988
4	Variables	Ernesto Villa	2018

Sintaxis

```
SELECT Nombre_libro,
autor_libro,
año_publicado
FROM Libros
GROUP BY autor_libro,
año_publicado,
nombre_libro
HAVING año_publicado < 2000;
```

Resultado

Nombre_libro	Autor_libro	Año_publicado
Algoritmos	Pedro Paz	1988

En el resultado podemos ver que se omiten registros ya que no satisfacen el HAVING condicional de haber sido publicados antes de 2000.

Quizás te preguntes por qué se necesitan dos cláusulas de filtrado diferentes: WHERE y HAVING.

Una cláusula WHERE no permite funciones agregadas en sus condicionales, un objetivo principal para la cláusula HAVING.

Por ejemplo, supongamos que deseamos comprobar qué valores de ID_libro existen más de una vez en nuestra tabla.

Nuestro primer intento de esto usando la cláusula GROUP BY podría verse así:

Sintaxis

```
SELECT id_libro
FROM Libros
WHERE COUNT(id_libro) > 1
GROUP BY id_libro;
```

Resultado

ERROR: aggregate functions are not allowed in WHERE

Como era de esperarse este error, por lo cual, en su lugar, usaremos el mismo condicional en la cláusula HAVING:

Sintaxis

```
SELECT id_libro
FROM Libros
GROUP BY id_libro
HAVING COUNT(id_libro) > 1;
```

Resultado

```
0
```

Esta vez nos dio el resultado correctamente ya que no hay duplicados en nuestra tabla.



CONSULTAS ESTATICAS

VIDEO

Te invitamos a vevr los siguientes videos:

UNIONES Y SUBCONSULTAS

JOINS

La consulta JOIN se utiliza en bases de datos para combinar dos o más tablas en una sola consulta. Las consultas JOIN son útiles para recuperar datos de varias tablas relacionadas. Existen varios tipos de JOIN, pero los más comunes son:

- Inner Join
- Left Join
- Right Join
- Full Outer Join

La unión de tablas para ver un conjunto de resultados no afecta a las tablas en ningún momento. No cambia nada físicamente en las tablas, respecto a su estructura o información.

La formación de la conexión implícita solo está dentro del tiempo de vida de la ejecución de la consulta de Join.

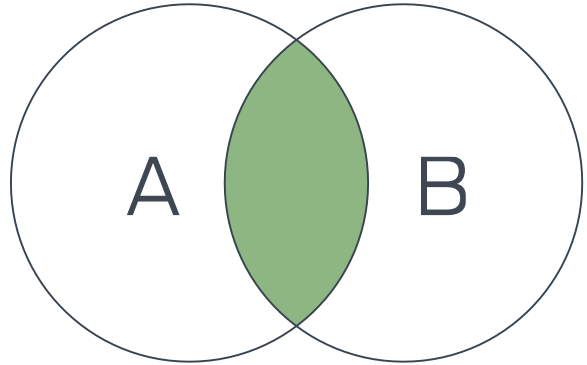


INNER JOIN

Devuelve solo las filas que tienen una coincidencia en ambas tablas. (Tabla 1 y Tabla 2).

Ejemplo: vemos la tabla Pedidos (Tabla 1):

ID_pedido	ID_cliente	Fecha_pedido
10	1	14-05-21
11	2	23-08-20
12	3	05-05-22
13	4	06-10-21



De igual forma, tenemos la tabla Clientes (tabla 2):

ID_cliente	Nombre_cliente	Ciudad
1	Alejandro Cruz	Tampico
2	Sandra Pérez	Mante
3	Pablo Torres	Valles
4	Mauricio Cruz	Madero

Sintaxis

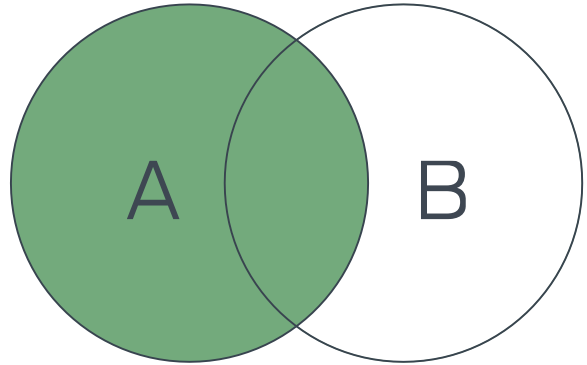
```
SELECT    Pedidos.ID_pedido,
Clientes.Nombre_cliente
FROM Pedidos
INNER JOIN Clientes
ON Pedidos.ID_pedido=Clientes.
ID_cliente;
```

Resultado

ID_pedido	Nombre_cliente
10	Alejandro Cruz
11	Sandra Pérez
12	Pablo Torres
13	Mauricio Cruz

LEFT JOIN

Devuelve todas las filas de la primera tabla (izquierda) y las filas coincidentes de la segunda tabla. El resultado es 0 registros del lado derecho, si no hay coincidencia.



Ejemplo: vemos la tabla Pedidos (tabla 1):

ID_pedido	ID_cliente	Fecha_pedido
10	1	14-05-21
11	2	23-08-20
12	3	05-05-22
13	4	06-10-21

De igual forma, tenemos la tabla Clientes (tabla 2):

ID_cliente	Nombre_cliente	Ciudad
1	Alejandro Cruz	Tampico
2	Sandra Pérez	Mante
3	Pablo Torres	Valles
4	Mauricio Cruz	Madero

Sintaxis

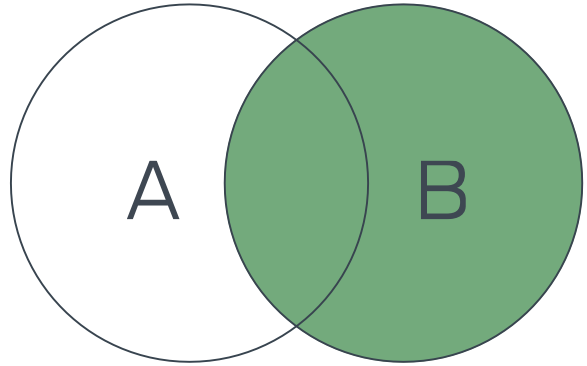
```
SELECT Clientes.Nombre_cliente, Pedidos.ID_pedido
FROM Clientes
LEFT JOIN Pedidos ON Clientes.ID_cliente = Pedidos.ID_cliente
ORDER BY Clientes.Nombre_cliente;
```

Resultado

Nombre_cliente	ID_pedido
Alejandro Cruz	
Mauricio Cruz	13
Pablo Torres	12
Sandra Pérez	

RIGHT JOIN

Devuelve todas las filas de la segunda tabla (derecha) y las filas coincidentes de la primera tabla. El resultado es 0 registros del lado izquierdo, si no hay ninguna coincidencia.



Ejemplo: vemos la tabla Pedidos (tabla 1):

ID_pedido	ID_cliente	Fecha_pedido
10	1	14-05-21
11	2	23-08-20
12	3	05-05-22
13	4	06-10-21

De igual forma, tenemos la tabla Clientes (tabla 2):

ID_cliente	Nombre_cliente	Ciudad
1	Alejandro Cruz	Tampico
2	Sandra Pérez	Mante
3	Pablo Torres	Valles
4	Mauricio Cruz	Madero

Sintaxis

```
SELECT Pedidos.ID_pedido,
Clientes.Nombre_cliente
FROM Clientes RIGHT JOIN
Pedidos
ON Clientes.ID_cliente=Pedidos.
ID_cliente
ORDER BY Pedidos.ID_pedido;
```

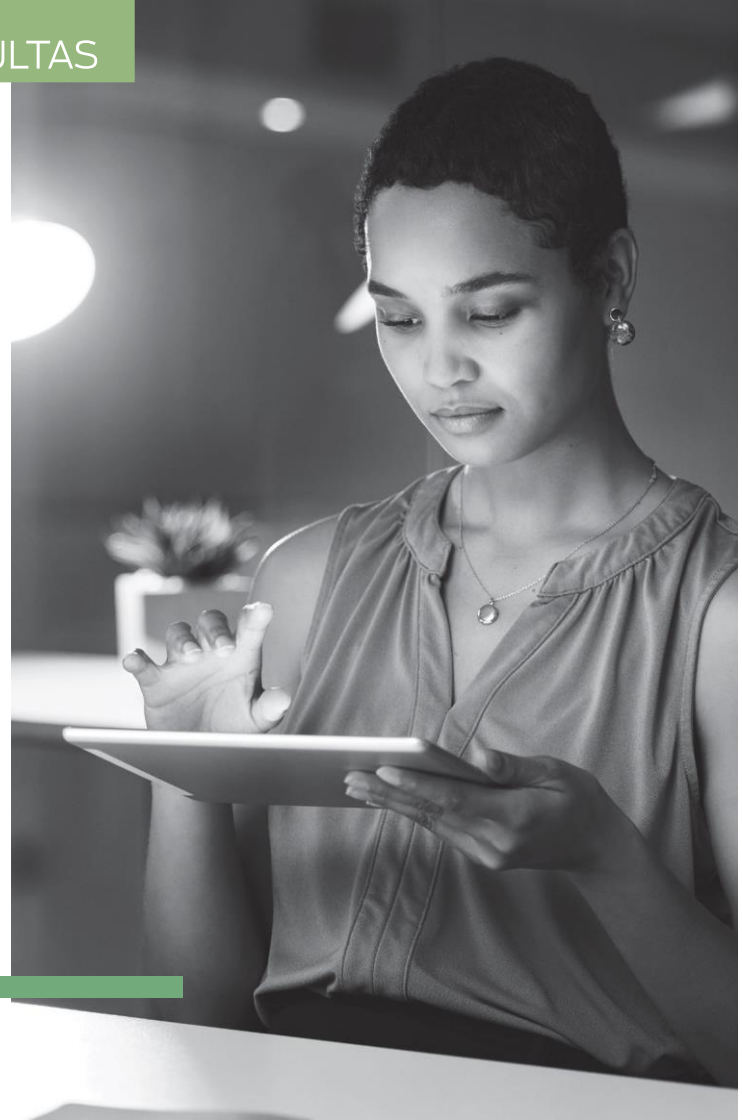
Resultado

ID_pedido	Nombre_cliente
10	Alejandro Cruz
11	Sandra Pérez
12	Pablo Torres
13	Mauricio Cruz

Subconsultas

Son consultas anidadas dentro de otra consulta. Las subconsultas se utilizan para obtener resultados más precisos y detallados al realizar una consulta. Estas pueden ser utilizadas en las cláusulas WHERE, HAVING y FROM.

Es como una instrucción SELECT dentro de otra. El resultado del INNER SELECT se puede usar en la consulta externa. La subconsulta siempre deberá ir entre paréntesis.



Ejemplo, tenemos la dos siguientes tablas:

Pedidos

ID_pedido	ID_cliente	Fecha_pedido
10	1	14-05-21
11	2	23-08-20
12	3	05-05-22
13	4	06-10-21

Clientes

ID_cliente	Nombre_cliente	Ciudad
1	Alejandro Cruz	Tampico
2	Sandra Pérez	Mante
3	Pablo Torres	Valles
4	Mauricio Cruz	Madero

Sintaxis

```
SELECT ID_pedido, ID_cliente
FROM Pedidos
WHERE ID_pedido IN (SELECT
ID_pedido FROM Pedidos);
```

Resultado

ID_pedido	ID_cliente
10	1
11	2
12	3
13	4



Tipos de subconsultas

Tenemos la posibilidad de clasificar las subconsultas en tres categorías.

1. Subconsultas escalares

Subconsulta que devuelve un solo valor. Esta subconsulta se utiliza en la cláusula WHERE, HAVING o SELECT de una consulta principal para filtrar o calcular valores.

2. Subconsultas de fila

Devuelven un conjunto de resultados con una sola columna y una sola fila. Estas subconsultas se utilizan principalmente para obtener un solo valor o una fila específica de una tabla y utilizarla en una consulta externa.

3. Subconsultas de tabla

Devuelven un conjunto de resultados que se pueden utilizar como una tabla en una consulta externa. Estas subconsultas se utilizan principalmente cuando se necesita obtener datos de una o varias tablas y utilizarlos en una consulta externa como si fueran una tabla.

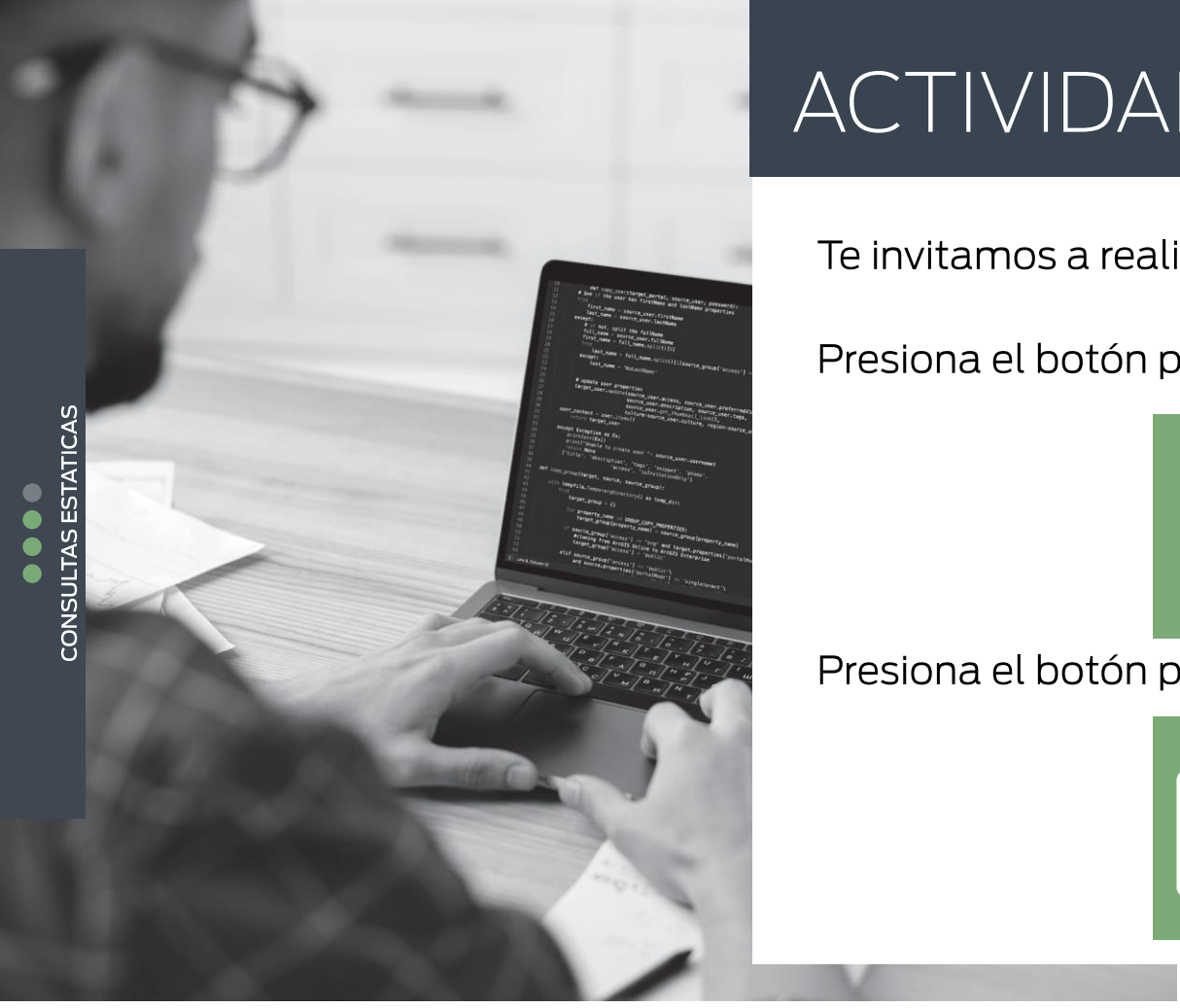




CONSULTAS ESTATICAS

VIDEO

Te invitamos a vevr los siguientes videos:



CONSULTAS ESTATICAS

ACTIVIDAD INTEGRADORA 2

Te invitamos a realizar la siguiente actividad.

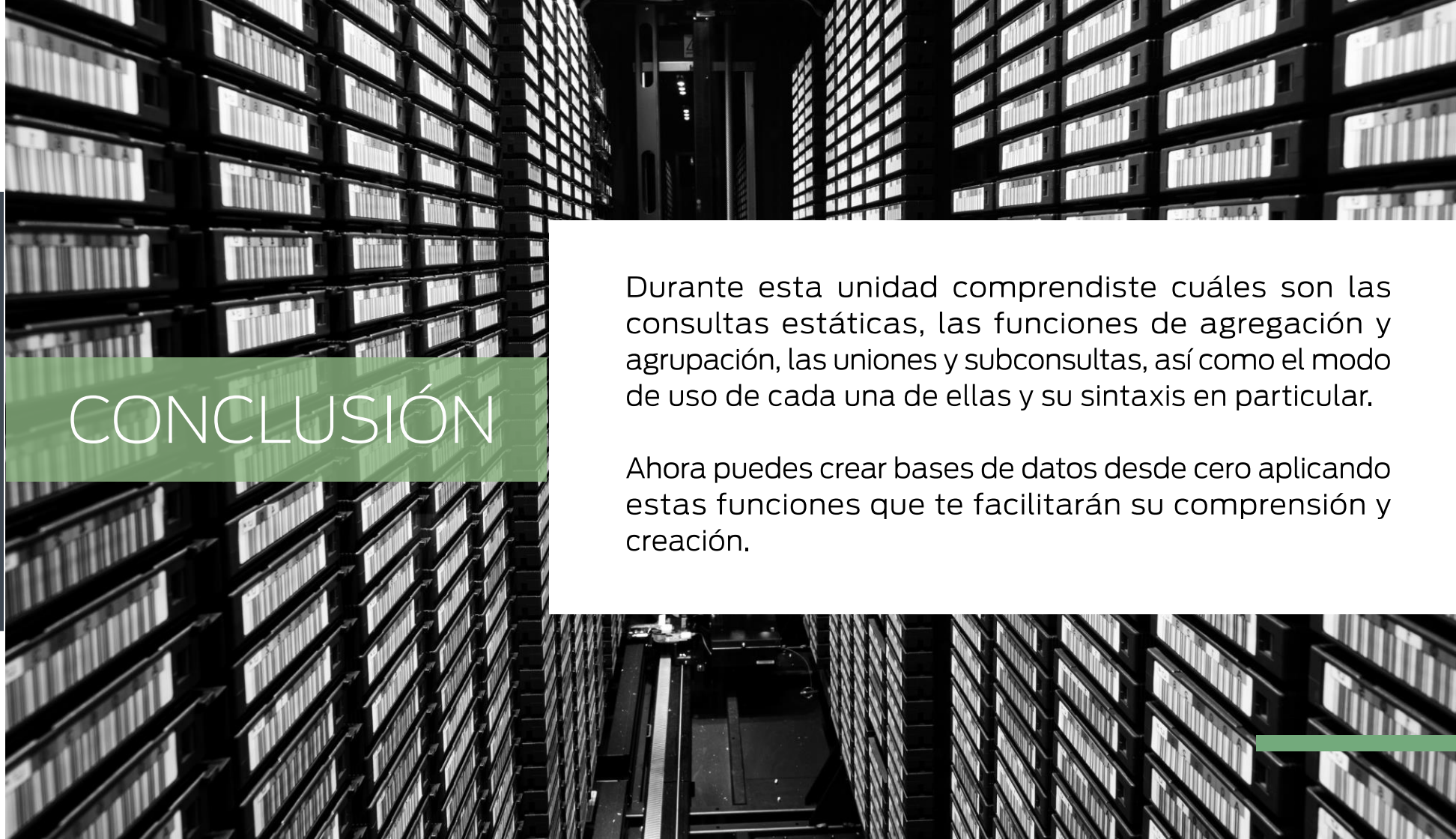
Presiona el botón para descargar la actividad.

Presiona el botón para entregar la actividad.

LIBROS RECOMENDADOS



- Cunningham, L. (2008). *The SQL Starter Series Book 1 -DML An Introduction to the SQL Data Manipulation Language.*



CONCLUSIÓN

Durante esta unidad comprendiste cuáles son las consultas estáticas, las funciones de agregación y agrupación, las uniones y subconsultas, así como el modo de uso de cada una de ellas y su sintaxis en particular.

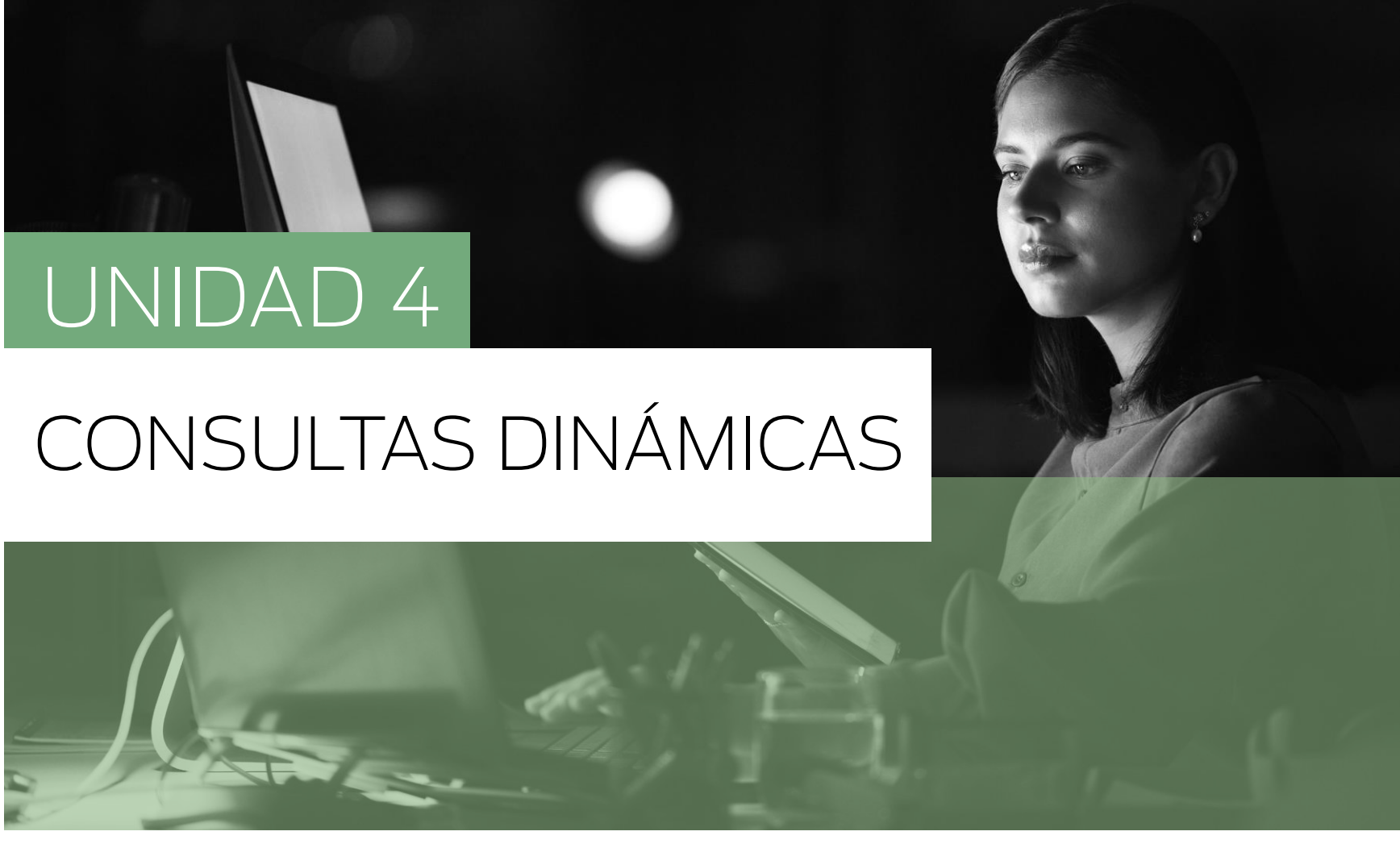
Ahora puedes crear bases de datos desde cero aplicando estas funciones que te facilitarán su comprensión y creación.



¡FELICIDADES!



Acabas de concluir la tercera unidad de tu curso **Introducción a las Bases de Datos**. Te invitamos a finalizar este esfuerzo realizando el examen parcial correspondiente. Para ello, debes regresar a la pantalla principal y dar clic en Presentar examen.



UNIDAD 4

CONSULTAS DINÁMICAS



TEMARIO

4.1 PARÁMETROS Y CICLOS

4.2 FUNCIONES TABULARES Y ESCALARES



INTRODUCCIÓN

En esta cuarta unidad aprenderás a crear parámetros y ciclos en SQL server de forma que te ayuden a ejecutar acciones de una manera más rápida.

A su vez, aprenderás a realizar funciones tabulares y escalares en tus bases de datos.

COMPETENCIAS A DESARROLLAR

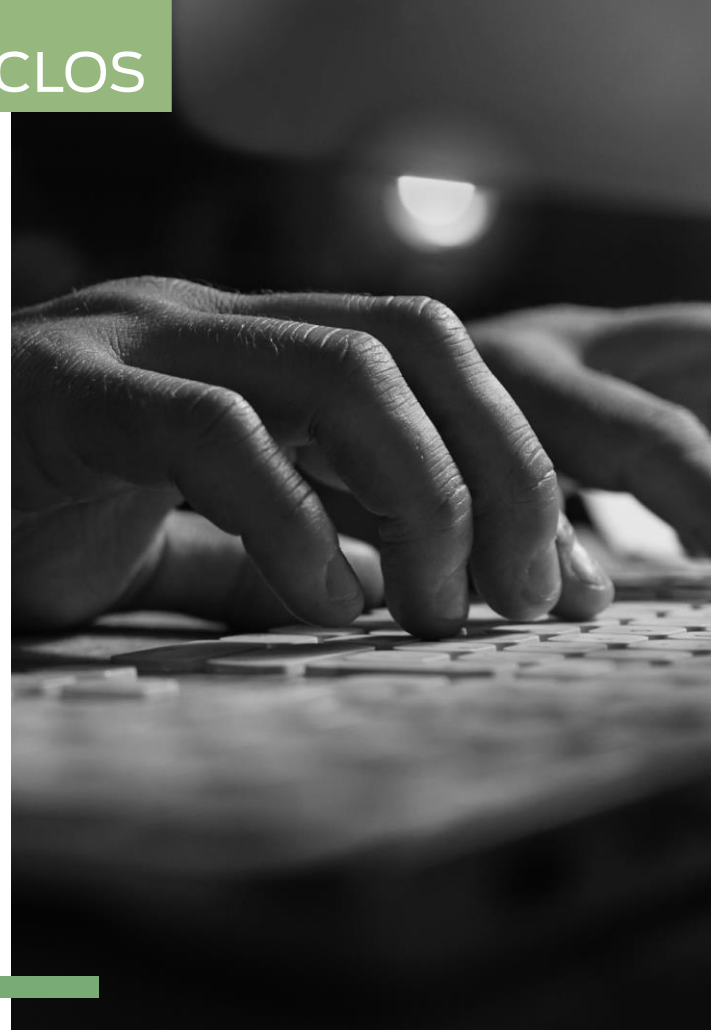


- El alumno será capaz de implementar parámetros y ciclos.
- El alumno será capaz de ejecutar funciones tabulares y escalares.

Consultas dinámicas

Es una forma de construir consultas personalizadas y flexibles que permiten al usuario especificar los criterios de búsqueda en tiempo de ejecución.

En una consulta dinámica, el usuario puede elegir qué campos de la base de datos desea buscar, qué valores deben tener esos campos, y cómo se deben combinar los criterios de búsqueda. En lugar de escribir una consulta fija que siempre busca los mismos datos, el usuario puede modificar la consulta según sus necesidades en ese momento.





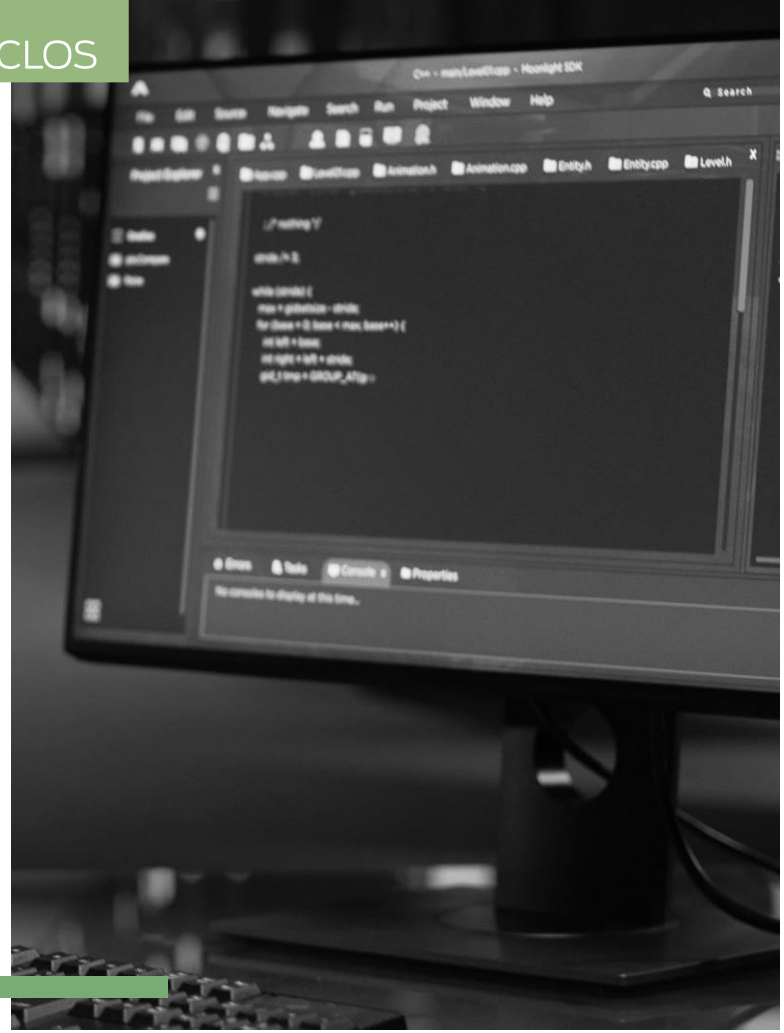
Procedimientos almacenados

Un procedimiento almacenado es una colección de declaraciones SQL que se guarda como un objeto en la base de datos y se puede invocar repetidamente para realizar una tarea específica.

Son muy útiles para reducir la cantidad de código que debe escribirse en una aplicación y mejorar el rendimiento de la base de datos. Además, pueden aceptar parámetros de entrada, realizar operaciones complejas y devolver resultados, lo que los hace muy versátiles y útiles en muchos contextos.

El proceso de un procedimiento almacenado en una base de datos generalmente sigue los siguientes pasos:

- **Creación:** El procedimiento almacenado se crea utilizando lenguaje SQL o una herramienta de desarrollo de base de datos. Durante la creación, se definen los parámetros de entrada, los parámetros de salida (si los hay) y el código SQL que se ejecutará cuando se llame al procedimiento almacenado.





- **Compilación:** Una vez que se crea el procedimiento almacenado, se compila para verificar que el código SQL sea sintácticamente correcto y para generar un plan de ejecución óptimo. El plan de ejecución se utiliza para optimizar la forma en que se ejecutará el procedimiento almacenado.

- **Almacenamiento:** Una vez que se compila el procedimiento almacenado, se almacena en la base de datos como un objeto. El objeto del procedimiento almacenado puede ser llamado y ejecutado por cualquier usuario con permisos suficientes.

- **Invocación:** Cuando se llama al procedimiento almacenado, se le pasan los parámetros de entrada necesarios. El procedimiento almacenado ejecuta el código SQL que se definió durante la creación, y puede modificar los datos en la base o devolver resultados a la aplicación que lo llamó.

- **Resultado:** Si el procedimiento almacenado devuelve un resultado, puede ser procesado por la aplicación que lo llamó. Si el procedimiento almacenado no devuelve un resultado, simplemente se completa la ejecución y los cambios que hizo en la base de datos permanecen guardados.





SQL Server agrega procedimientos almacenados del sistema, están en la base de datos principal y se reconocen por su nombre, todos tienen un nombre que comienza con SP_ (prefijo de procedimientos almacenados).

Le permiten restaurar información de las tablas del sistema, y se pueden ejecutar en cualquier base de datos del servidor.

El mantenimiento del procedimiento se genera resolviendo el procedimiento de creación y lo realiza Exec (o Execute).

La sentencia “DROP PROCEDURE” es una instrucción SQL utilizada para eliminar un procedimiento almacenado existente en una base de datos. La sintaxis general de la sentencia es la siguiente:

```
DROP PROCEDURE nombre_prodecimiento;
```

Donde “nombre_procedimiento” es el nombre del procedimiento almacenado que se desea eliminar.

Es importante tener en cuenta que una vez que se elimina un procedimiento almacenado, no se puede recuperar, por lo que es necesario tener cuidado al utilizar la sentencia “DROP PROCEDURE”.

Es recomendable tener una copia de seguridad de la base de datos antes de eliminar cualquier objeto de la misma.

Ejemplo:

Tenemos creada la base de datos *Tienda*, donde existe la base de datos Clientes, por lo que el procedimiento almacenado se creará de la siguiente forma:

Sintaxis

```
CREATE PROCEDURE TodosLosClientes
AS
SELECT * FROM Clientes
GO;
exec TodosLosClientes
```

Utilizamos *exec* para ejecutar nuestro procedimiento almacenado.

Parámetros

Es un valor que se utiliza como entrada o salida en una consulta o en un procedimiento almacenado. Un parámetro puede ser utilizado para filtrar resultados de una consulta, especificar valores para la actualización de una tabla o para la ejecución de un procedimiento almacenado.

Los parámetros se utilizan para proporcionar información dinámica a las consultas y a los procedimientos almacenados en lugar de requerir que se escriba información estática directamente en la consulta o en el procedimiento almacenado.



Ejemplo:

Retomando el ejemplo del Procedimiento almacenado, tenemos la tabla *Clientes*, por lo que se utiliza un parámetro de la siguiente forma:

```
CREATE PROCEDURE TodosLosClientes @Ciudad nvarchar(30)
AS
SELECT * FROM Clientes WHERE Ciudad = @Ciudad;

EXEC TodosLosClientes @Ciudad = 'Tampico';
```

Donde, *@Ciudad* es el parámetro creado, donde un procedimiento almacenado llamado *TodosLosClientes*, selecciona a los Clientes de la ciudad Tampico, de la tabla Clientes.



Parámetros de entrada

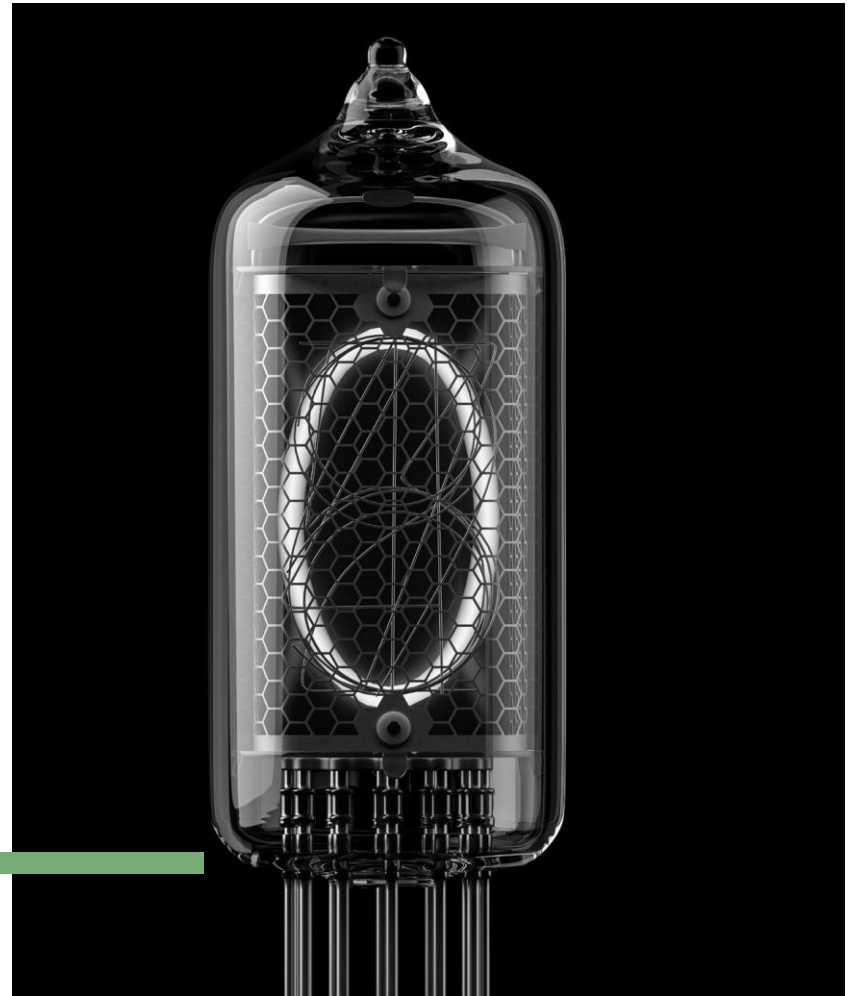
- Valor que se utiliza como entrada en una consulta o en un procedimiento almacenado. Permite a los usuarios o a las aplicaciones proporcionar información dinámica a una consulta o a un procedimiento almacenado en tiempo de ejecución.

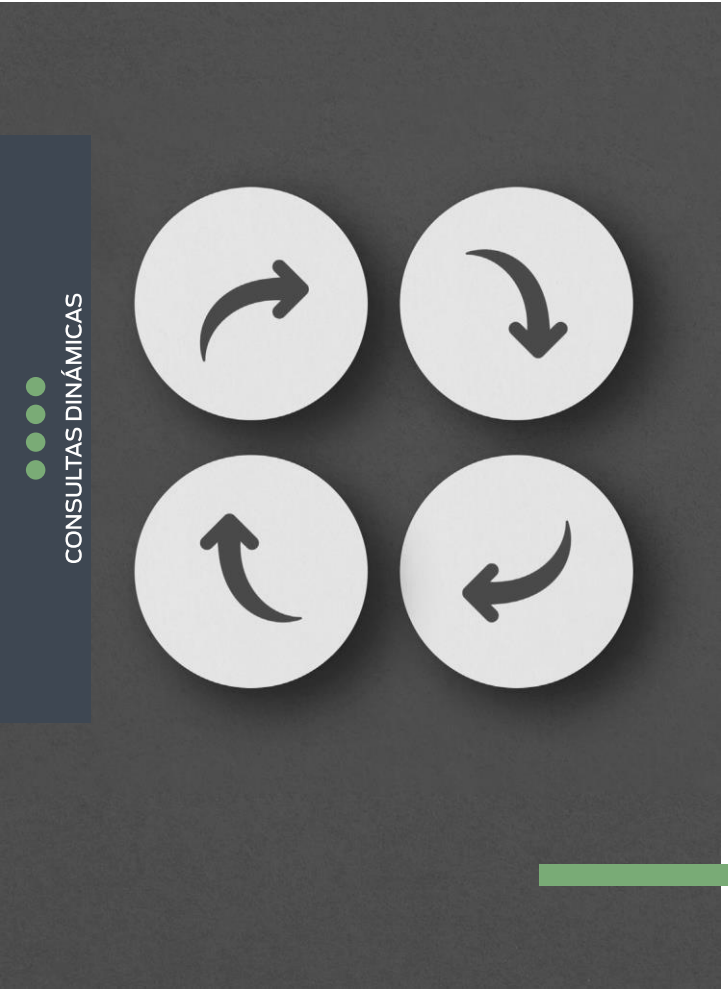
Parámetros de salida

- Valor que se utiliza como salida en un procedimiento almacenado. Permiten que un procedimiento almacenado proporcione información específica después de que se haya completado su ejecución.

Cada procedimiento guardado devuelve un código de retorno completo al que hace una llamada.

Si el procedimiento preservado claramente no establece el valor para el código de retorno, esto es 0.



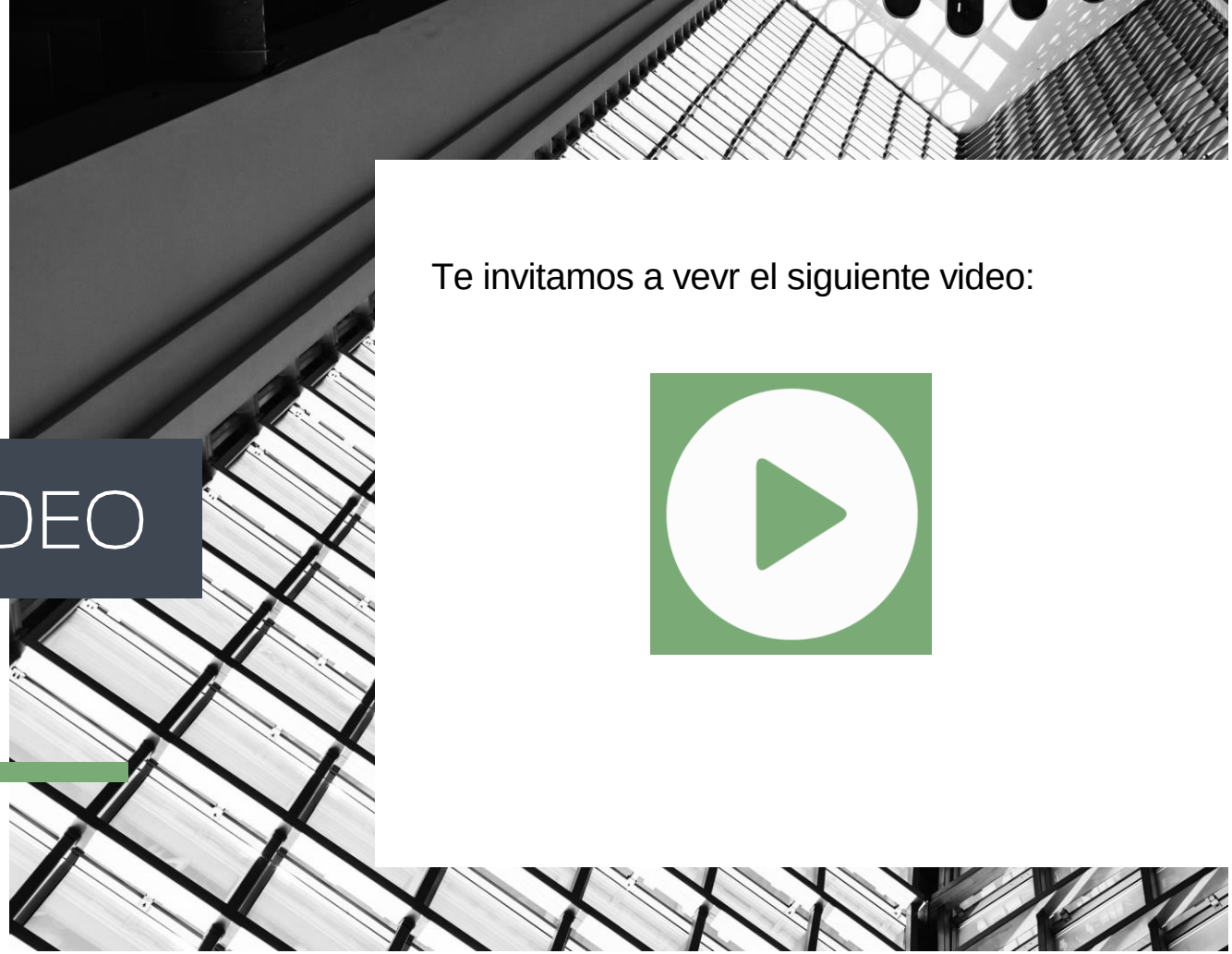


Ciclos

Estructuras de control que se utilizan para repetir una acción o un conjunto de acciones en una tabla o conjunto de tablas. Los ciclos son importantes en la programación de bases de datos, ya que permiten automatizar tareas repetitivas y procesar grandes cantidades de datos.

Hay varios tipos de ciclos que se pueden utilizar en una base de datos, dependiendo del tipo de tarea que se quiera realizar. Algunos de los ciclos más comunes son:

- Ciclo WHILE
- Ciclo IF ELSE



CONSULTAS DINÁMICAS

VIDEO

Te invitamos a vevr el siguiente video:

Ciclo WHILE en SQL SERVER
Se utiliza para repetir un bloque de instrucciones mientras se cumpla una condición específica. La sintaxis básica del ciclo WHILE en SQL Server es la siguiente:

```
WHILE (condición)
BEGIN
  -- bloque de instrucciones
END
```

La condición es una expresión lógica que se evalúa en cada iteración del ciclo. Si la condición es verdadera, el bloque de instrucciones se ejecuta una vez más. Si la condición es falsa, el ciclo se detiene y el control se pasa a la siguiente instrucción después del bloque WHILE.

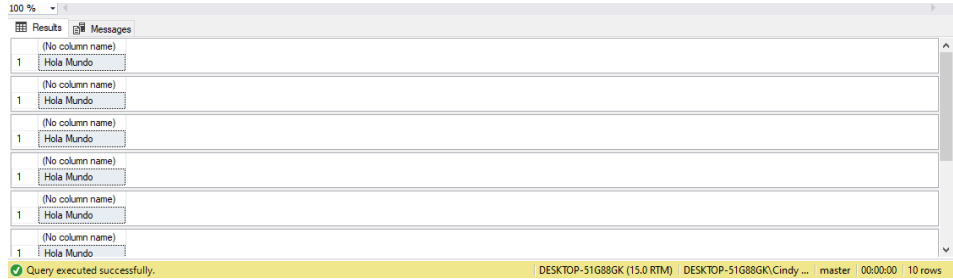
PARÁMETROS Y CICLOS

Ejemplo:

Sintaxis

```
DECLARE @Cont INT = 1;
WHILE(@Cont <= 10) BEGIN
   select 'Hola Mundo';
   set @Cont += 1;
END
```

Resultado



Ciclo IF ELSE en SQL SERVER

IF ELSE se utiliza para tomar decisiones basadas en una condición.

Sintaxis

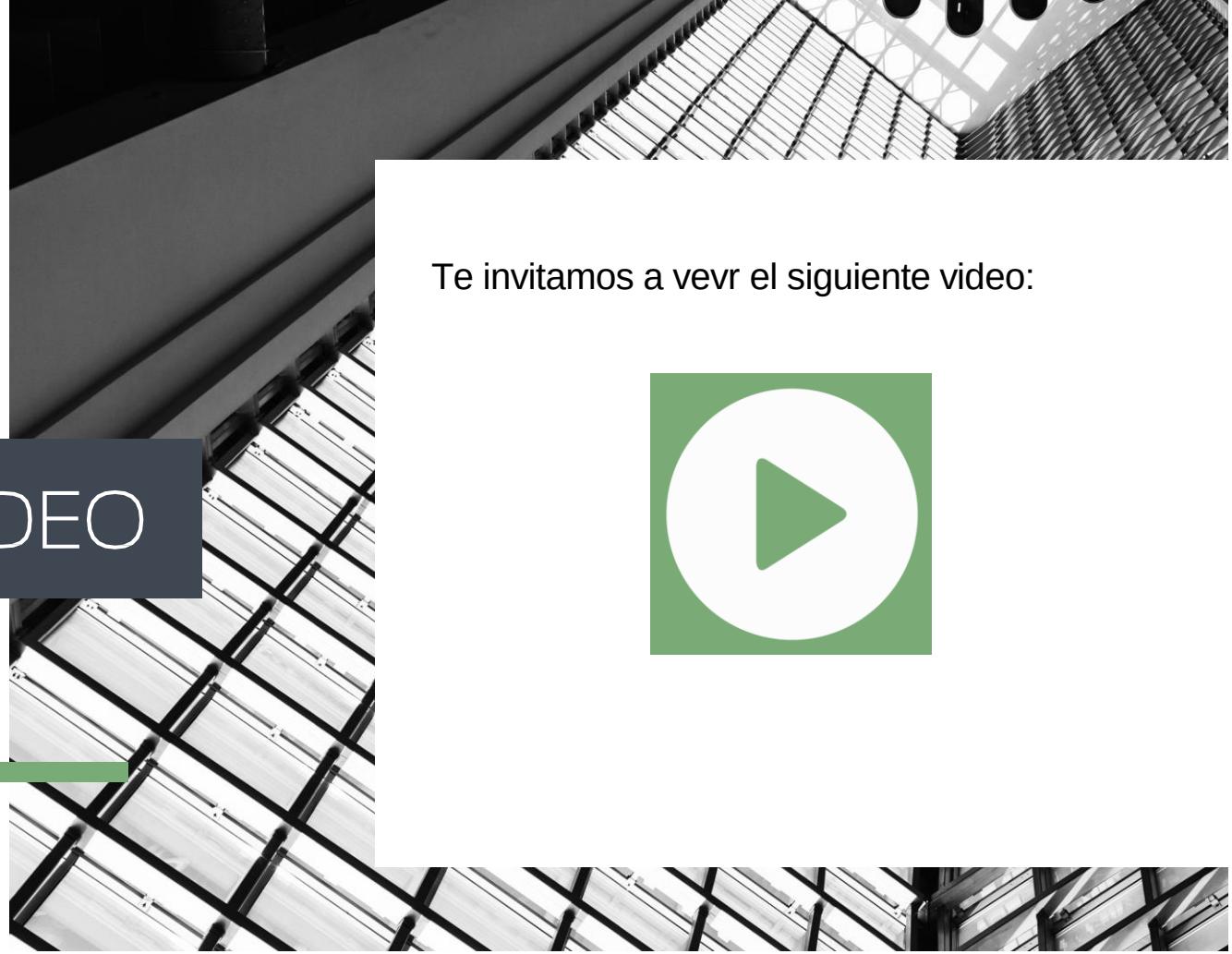
```
DECLARE @Tabla INT = 1;
IF(@Tabla = 1)
   SELECT * FROM  Pedidos
ELSE
   SELECT * FROM  Clientes
```

```
IF condición
BEGIN
    -- acciones a realizar si la condición es verdadera
END
ELSE
BEGIN
    -- acciones a realizar si la condición es falsa
END
```

La palabra clave **BEGIN** se utiliza para indicar el inicio de un bloque de código y **END** para indicar el final del bloque de código. En el siguiente ejemplo, se muestra cómo utilizar el ciclo IF ELSE en SQL Server para realizar una consulta que devuelva un mensaje si el valor de una columna es mayor a un valor específico:

```
DECLARE @valor int;
SET @valor = 10;

IF (SELECT columna FROM tabla WHERE condición) > @valor
BEGIN
    SELECT 'La columna es mayor a ' + CONVERT(varchar(10), @valor);
END
ELSE
BEGIN
    SELECT 'La columna es menor o igual a ' + CONVERT(varchar(10), @valor);
END
```



CONSULTAS DINÁMICAS

VIDEO

Te invitamos a vevr el siguiente video:

FUNCIONES TABULARES Y ESCALARES

Funciones tabulares

Una función tabular es una función que devuelve un conjunto de filas (o tabla) como resultado. A diferencia de las funciones escalares, que devuelven un solo valor, las funciones tabulares se utilizan para realizar cálculos y consultas complejas que devuelven un conjunto de resultados. Existen dos tipos de funciones tabulares en SQL Server:

Funciones de tabla inline (inline table-valued functions, iTVFs)
Funciones de tabla multilínea (multi-statement table-valued functions, mTVFs)

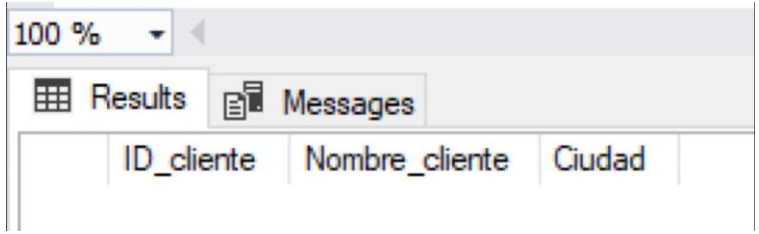
Ejemplo para ordenar a los Clientes por Nombre del cliente:

```
SET ANSI_NULLS ON
GO
SET QUOTED_IDENTIFIER ON
GO

CREATE FUNCTION CLIENTES_POR_NOMBREDELCLIENTE
(
   @Valor varchar
)
RETURNS TABLE
AS
RETURN
(
   SELECT * FROM Clientes WHERE Nombre_cliente = @Valor
)
GO
```

Para ejecutar la función:
SELECT * FROM CLIENTES_POR_NOMBREDELCLIENTE(1);

Resultado



Funciones de tabla inline (inline table-valued functions, iTVFs): Estas funciones se definen utilizando la cláusula RETURNS TABLE y el cuerpo de la función está compuesto por una única sentencia SELECT. Las funciones iTVFs se pueden utilizar como si fueran tablas, y se pueden incluir en una consulta como una tabla normal. Veamos un ejemplo de una función iTVF que devuelve una tabla de empleados cuyo salario es mayor que una cantidad dada:

```
CREATE FUNCTION fn_empleados_salario_superior(@salario_minimo INT)
RETURNS TABLE
AS
RETURN
(
    SELECT *
    FROM Empleados
    WHERE Salario > @salario_minimo
)
```

Funciones de tabla multilínea:

Estas funciones se definen utilizando la cláusula RETURNS @table_ variable y el cuerpo de la función está compuesto por múltiples sentencias SQL. Las funciones mTVFs son similares a las iTVFs, pero son más flexibles y se pueden utilizar para realizar cálculos más complejos. A continuación, se muestra un ejemplo de una función mTVF que devuelve una tabla de empleados cuyo salario está entre dos cantidades dadas:

```
CREATE FUNCTION fn_empleados_salario_entre(@salario_minimo INT, @salario_maximo INT)
RETURNS @empleados_salario_entre TABLE
(
    EmpleadoID INT,
    Nombre NVARCHAR(50),
    Salario INT
)
AS
BEGIN
    INSERT INTO @empleados_salario_entre
    SELECT EmpleadoID, Nombre, Salario
    FROM Empleados
    WHERE Salario BETWEEN @salario_minimo AND @salario_maximo

    RETURN
END
```

Funciones escalares

Función que devuelve un solo valor como resultado. Estas funciones se utilizan para realizar cálculos o procesamiento de datos más complejos y devolver un resultado que se puede utilizar en una consulta.

Existen dos tipos de funciones escalares en SQL Server:

Funciones escalares definidas por el usuario (user-defined scalar functions, UDFs)
Funciones escalares integradas en el sistemamTVFs)

Ejemplo para calcular la raíz cuadrada de 99:

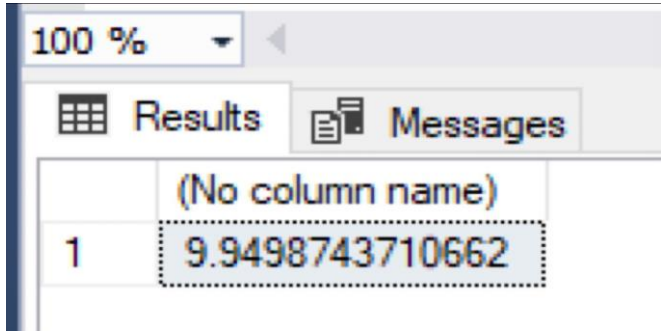
```
SET ANSI_NULLS ON
GO
SET QUOTED_IDENTIFIER ON
GO

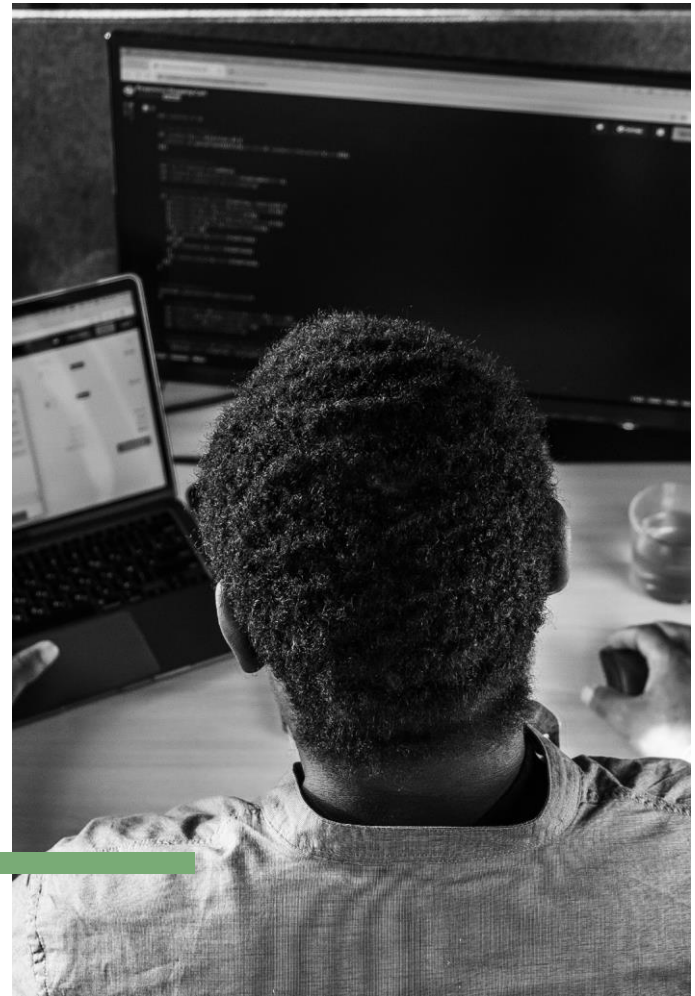
CREATE FUNCTION CALCULAR_
RAIZ_CUADRADA
(
   @VALOR FLOAT
)
RETURNS FLOAT
AS
BEGIN
      DECLARE  @RESULTADO
FLOAT;
   SET @RESULTADO = SQRT(@
VALOR);
   RETURN @RESULTADO;
END
GO
```

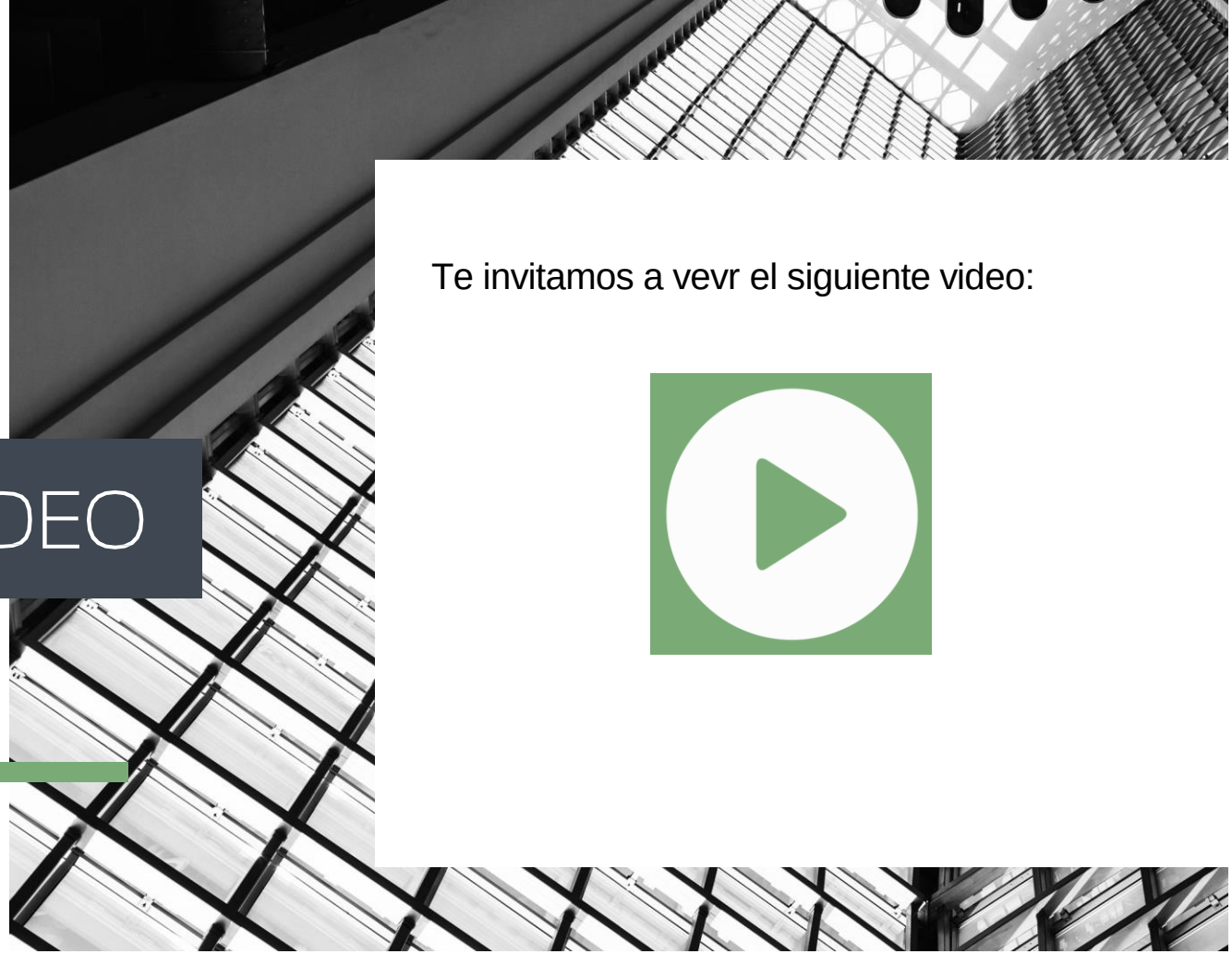
Para ejecutar la función:

SELECT dbo.CALCULAR_RAIZ_CUADRADA(99);

Resultado







CONSULTAS DINÁMICAS

VIDEO

Te invitamos a vevr el siguiente video:



Funciones escalares definidas por el usuario:

Estas funciones se definen por el usuario y se pueden utilizar en una consulta como si fueran una función incorporada en el sistema. Las funciones UDFs se definen utilizando la cláusula RETURNS seguida del tipo de datos del valor devuelto. Veamos un ejemplo de una función UDF que devuelve el área de un círculo a partir del radio:

En este ejemplo, la función UDF CalcularAreaCirculo recibe un parámetro @radio de tipo FLOAT y devuelve un valor FLOAT que representa el área del círculo. La función utiliza la constante PI() y la función POWER() para realizar los cálculos y devuelve el resultado mediante la instrucción RETURN.

```
CREATE FUNCTION dbo.CalcularAreaCirculo(@radio FLOAT)
RETURNS FLOAT
AS
BEGIN
    DECLARE @area FLOAT
    SET @area = PI() * POWER(@radio, 2)
    RETURN @area
END
```

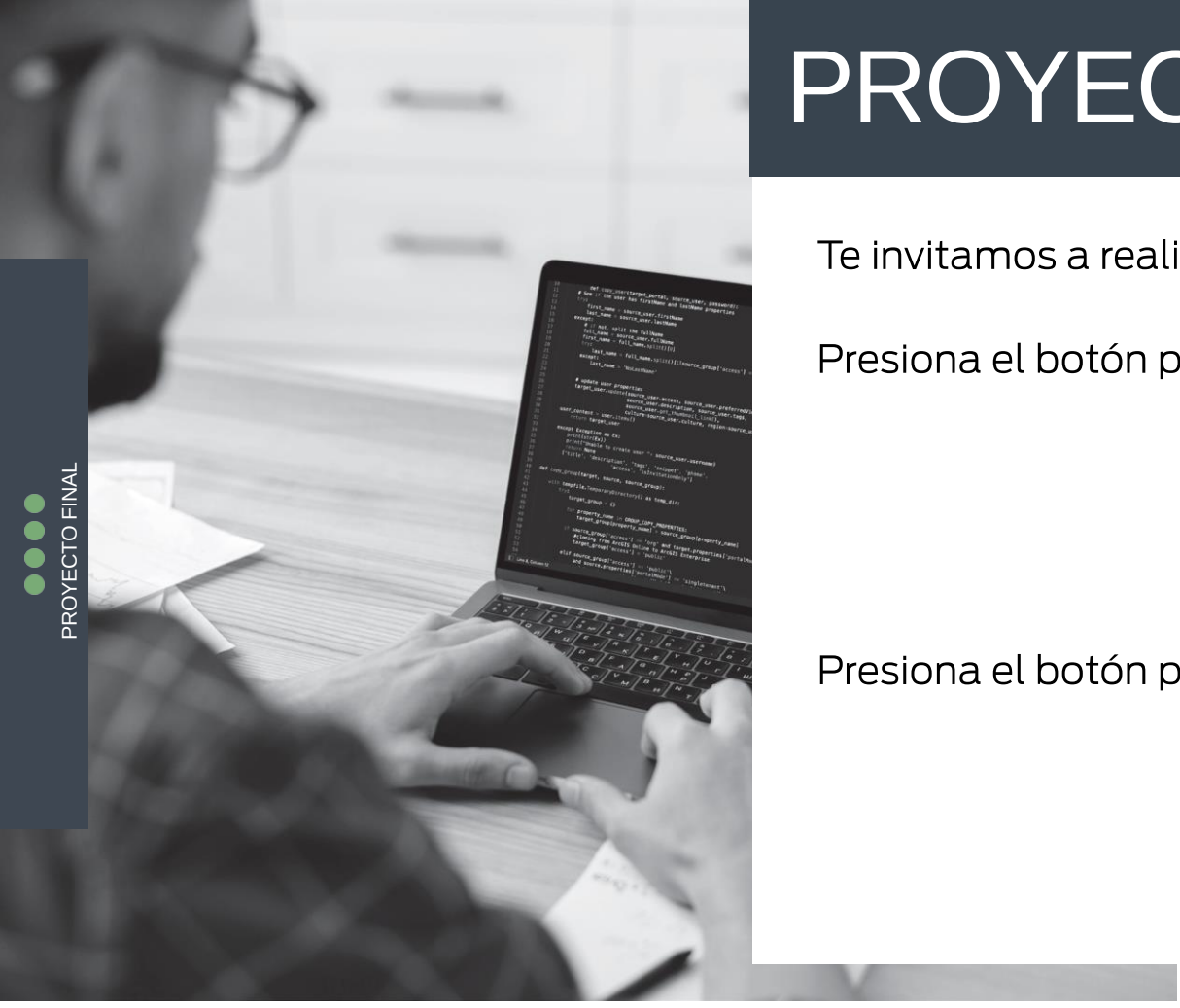
Funciones escalares integradas en el sistema:

SQL Server incluye varias funciones escalares integradas en el sistema que se pueden utilizar en una consulta. Estas funciones incluyen operaciones matemáticas básicas, funciones de cadena, funciones de fecha y hora, entre otras. Veamos un ejemplo de una función integrada en el sistema que devuelve la longitud de una cadena:

```
SELECT LEN('Hola, mundo!')
```

En este ejemplo, la función integrada LEN() se utiliza para devolver la longitud de la cadena “Hola, mundo!”.

PROYECTO FINAL



PROYECTO FINAL

PROYECTO FINAL

Te invitamos a realizar la siguiente actividad.

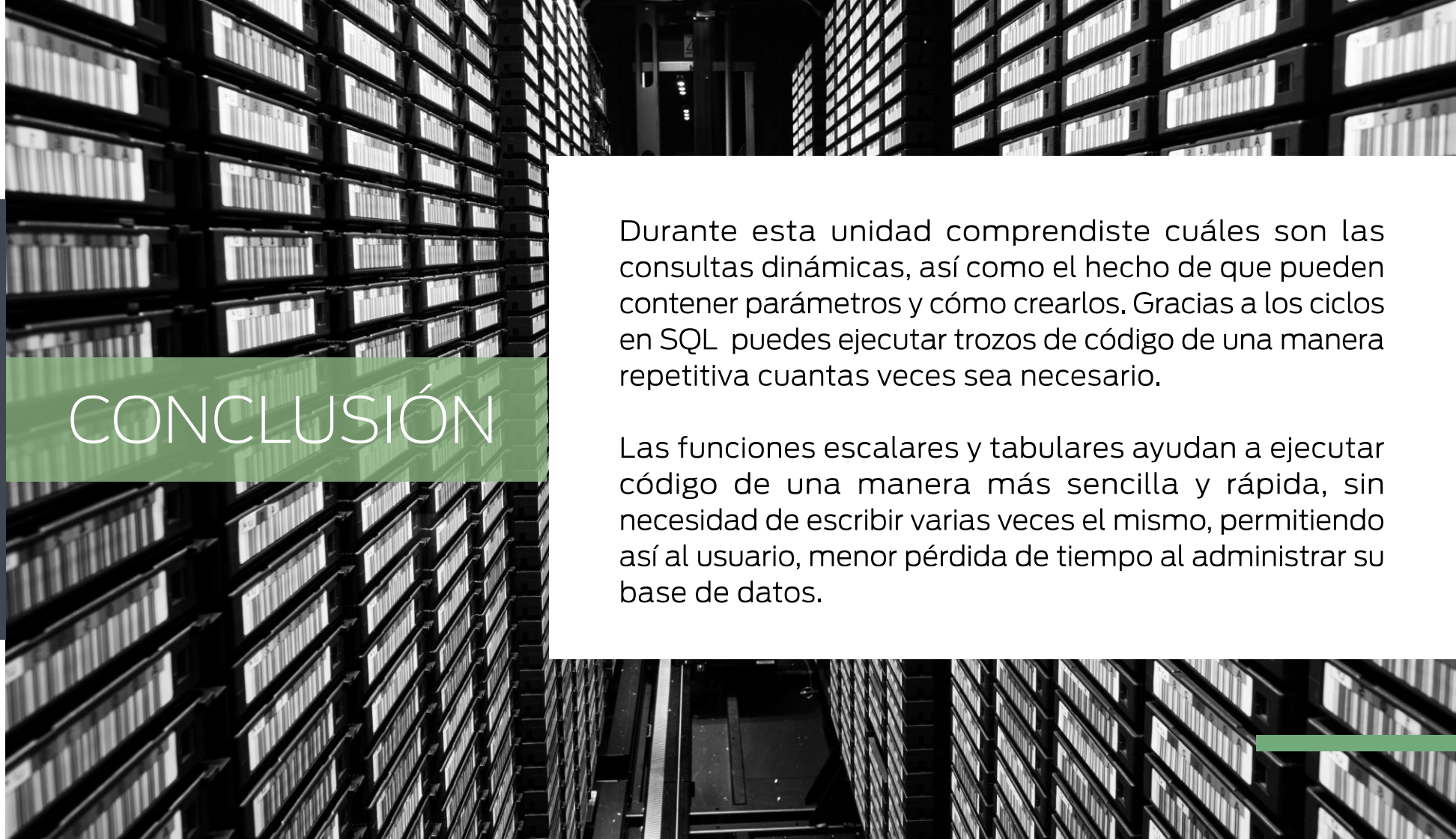
Presiona el botón para descargar la actividad.

Presiona el botón para entregar la actividad.

LIBROS RECOMENDADOS



- Liliana, O. (2022). *Análisis comparativo de rendimiento en gestores de bases de datos relacionales y no relacionales.*
- Araneda, P. (2022, 18 octubre). Capítulo 6 Normalización. *Base de Datos.*
- Pulido, E., Escobar, Óscar., Núñez, J. (2019). *Base de datos.*
- aulaClic, Curso de SQL Server.



CONCLUSIÓN

Durante esta unidad comprendiste cuáles son las consultas dinámicas, así como el hecho de que pueden contener parámetros y cómo crearlos. Gracias a los ciclos en SQL puedes ejecutar trozos de código de una manera repetitiva cuantas veces sea necesario.

Las funciones escalares y tabulares ayudan a ejecutar código de una manera más sencilla y rápida, sin necesidad de escribir varias veces el mismo, permitiendo así al usuario, menor pérdida de tiempo al administrar su base de datos.

CONSULTAS DINÁMICAS



¡FELICIDADES!

Acabas de concluir la cuarta unidad de tu curso **Introducción a las Bases de Datos**. Te invitamos a finalizar este esfuerzo realizando el examen parcial correspondiente. Para ello, debes regresar a la pantalla principal y dar clic en Presentar examen.